TCS 704 — Advanced Computer Architecture

Detailed Study Notes — Unit 1 & Unit 2 (Illustrated Edition)

B.Tech CSE, Semester VII

Subject Code	TCS 704	**Credits**	3
L-T-P	3-0-0	**Category**	DSC
Pre-requisite	Logic Design and Computer Organization (TCS308)		
Unit 1	Basics of Computing (9 Contact Hrs)		
Unit 2	Memory Hierarchy Design (10 Contact Hrs)		
Key Text	Kai Hwang, "Advanced Computer Architecture", 3rd Ed., McGraw Hill		
Reference	Hennessy & Patterson, "Computer Architecture: A Quantitative Approach"		

Contents

UNIT 1 — Basics of Computing

1. What is Computer Architecture?
2. Technology Trends & Moore's Law
3. Classification of Parallel Computers (Flynn's Taxonomy)
4. Performance-Based Computing & the CPU Performance Equation
5. The "Myopic View" of Computer Architecture
6. Energy, Power and Cost Trends
7. Dependability
8. Processor Speed, Cost, Power Consumption & Fabrication Yield
9. Performance Metrics, Benchmarks, Amdahl's Law & Gustafson's Law

UNIT 2 — Memory Hierarchy Design

1. Basics of Memory Hierarchy
2. Inclusion, Coherence and Locality Properties
3. Cache Memory Organization
4. Advanced Optimization of Cache Performance
5. Memory Technology and Optimization
6. Cache Coherence and Synchronization Mechanisms
7. Virtual Memory
8. Virtual Machines

UNIT 1 : Basics of Computing

1.1 What is Computer Architecture?

Computer Architecture is not just about the instruction set — it is the science of designing the structure and behaviour of a computer system so that it meets functional, performance, cost, and power goals simultaneously. Early textbooks equated "architecture" with only the instruction set, but modern usage recognises that delivered performance emerges from an entire stack of decisions, from the application down to the transistor. Architects must reason across this whole stack because a change at one layer (say, a smarter compiler) can substitute for — or completely undo — an expensive change at another layer (say, a wider pipeline).

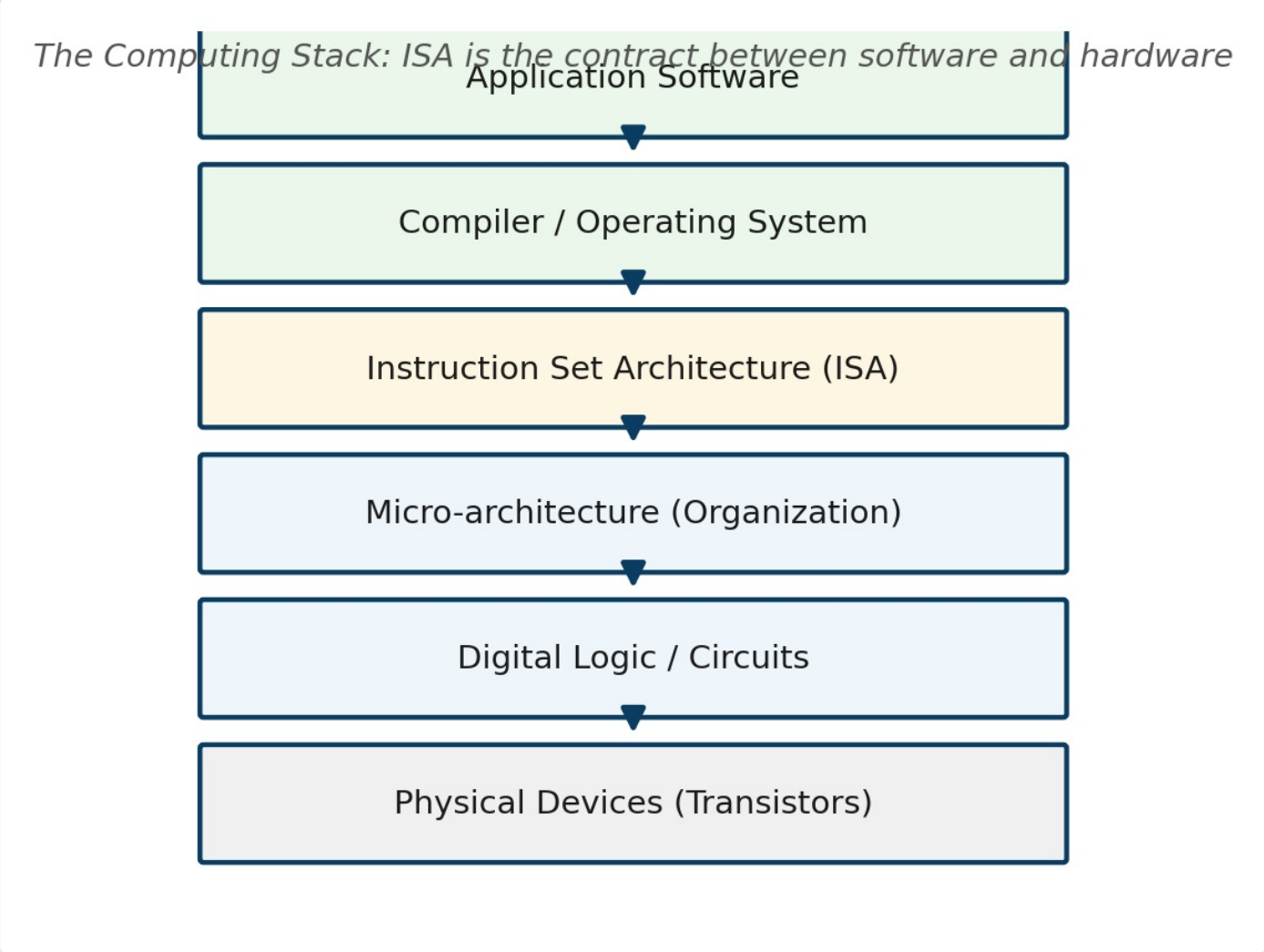


Fig 1.1 — The computing stack: each layer is built upon, and abstracts away, the one below it. The ISA is the fixed contract that lets software and hardware evolve independently.

The three commonly separated layers are:

Instruction Set Architecture (ISA): the interface between hardware and software — the set of opcodes, addressing modes, registers, and data types visible to a programmer/compiler. It is what a compiler writer or assembly programmer "sees."

Computer Organization (Micro-architecture): how the ISA is actually implemented internally — pipeline depth, cache sizes, number of ALUs, branch predictor design, out-of-order issue logic, etc. This is invisible to the programmer but decides performance.

Hardware / Implementation: the actual logic design, chip layout (floor-planning), and circuit/fabrication technology used to physically realize the organization (e.g., 7 nm vs 5 nm process).

Two computers can share the same ISA (e.g., both are "x86-64" machines) but have completely different organizations (an Intel Atom low-power core vs. an Intel Core i9 high-performance core), giving very different performance, power, and cost characteristics while running the exact same compiled binary. This separation of concerns — one ISA, many possible organizations — is what has allowed instruction sets like x86 and ARM to survive for decades while the underlying hardware has been completely redesigned dozens of times.

RISC vs CISC — a related ISA-design classification

Aspect	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)

Instruction complexity	Many instructions, variable length, can do multi-step operations (e.g., memory-to-memory add)	Few, simple, fixed-length instructions; mostly register-to-register
CPI	Higher CPI, but fewer instructions per program	Aims for CPI close to 1; needs more instructions per program
Hardware	Complex control unit, often microcoded	Simple, hard-wired control; easier to pipeline
Example ISAs	x86 (Intel/AMD)	ARM, RISC-V, MIPS, older SPARC/PowerPC

Why study architecture now?

For decades single-processor performance grew almost automatically thanks to faster clocks, deeper pipelines and cleverer out-of-order execution. That "free lunch" ended around 2004 because of the power wall (Section 1.6), pushing the whole industry toward multicore and parallel designs. That single historical turning point is the reason this syllabus places "classification of parallel computers" and, later, an entire unit on multiprocessor memory hierarchy so early in the course — parallelism is now the default way to get more performance, not an exotic option.

1.2 Technology Trends and Moore's Law

Moore's Law

> **Statement (Gordon Moore, 1965, revised 1975):** The number of transistors that can be economically placed on an integrated circuit doubles approximately every 18–24 months.

Moore's Law is *not* a law of physics — it is an empirical observation about manufacturing economics that became a self-fulfilling industry roadmap. Semiconductor companies (and the collective "International Technology Roadmap for Semiconductors") planned billions of dollars of R&D specifically to keep pace with the doubling trend, which is why it held so precisely for almost 50 years.

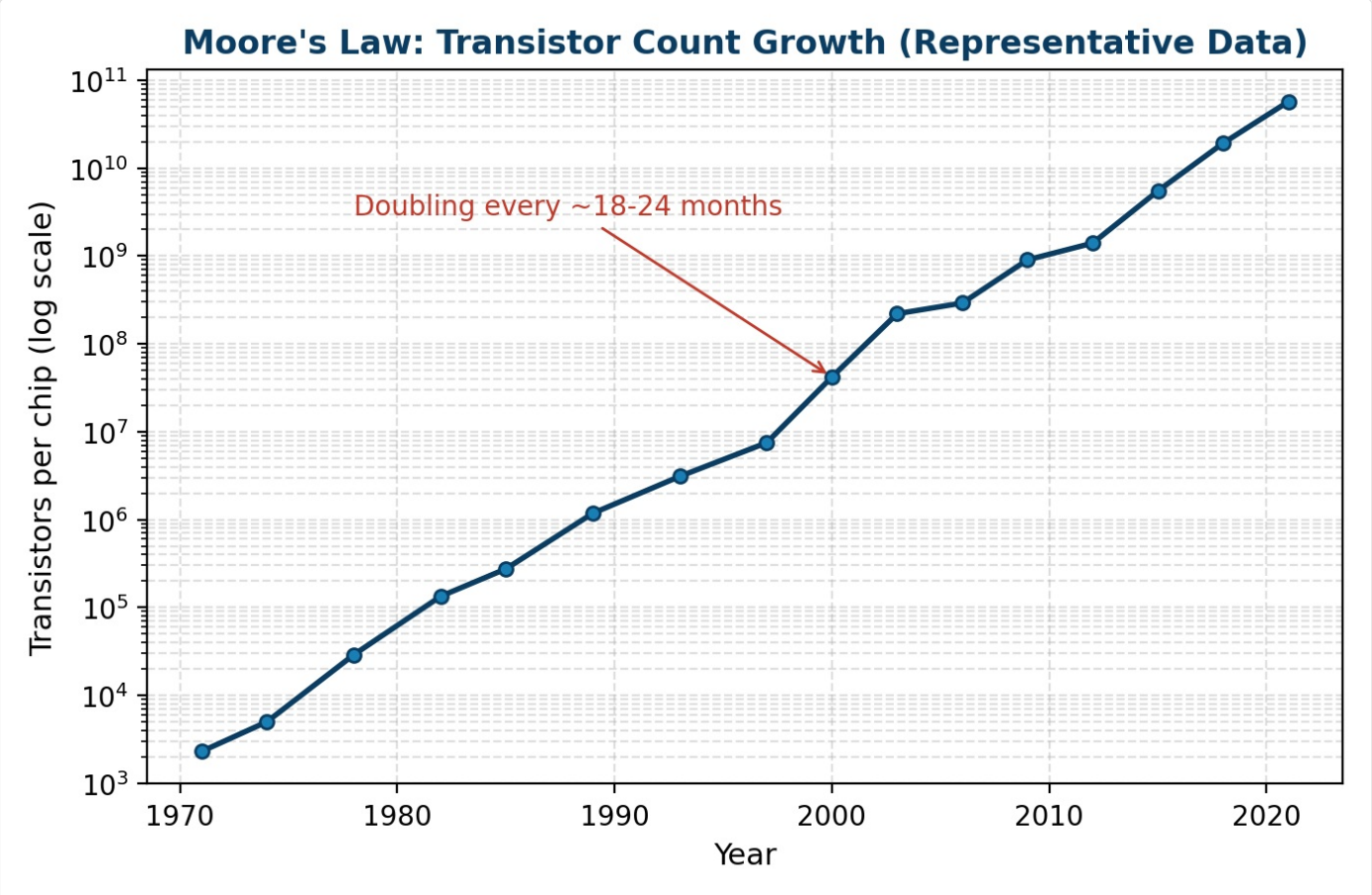


Fig 1.2 — Representative growth in transistor count per chip since 1971 (log scale). Notice the almost-straight line on a log axis, which is the visual signature of exponential (doubling) growth.

Applications / Consequences of Moore's Law

- **More transistors → more functionality** per chip: bigger caches, more execution units, more cores, on-chip memory controllers/GPUs.
- **Cost per transistor falls** even as raw chip cost per unit area stays roughly constant, driving down the price of computing power over time — this is why a smartphone today has vastly more compute than a 1990s supercomputer, at a tiny fraction of the price.
- Enabled the historical shift: single-core scalar CPUs → pipelined CPUs → superscalar (multiple-issue) CPUs → multicore → many-core / heterogeneous (CPU+GPU) designs, because architects had "more transistors than they knew what to do with" at each stage and had to find new ways to spend the budget.
- **Its slow-down** (transistor scaling has become physically harder and slower below ~10 nm due to quantum tunnelling and heat-dissipation limits) is a major reason parallel and heterogeneous computing became necessary — you can no longer simply wait 18 months for a free doubling of single-core speed.

Broader Technology and Performance-Growth Eras

Hennessy & Patterson identify distinct eras of computer performance growth, driven by different underlying factors:

Era	Approx. Period	Main Performance Driver	Growth Rate
Pre-RISC	1970s – 1985	Semiconductor + architecture improvements together	~25%/yr
RISC era	1985 – 2004	Instruction-level parallelism, pipelining, caches, clock scaling	~35–52%/yr
Multicore era	2004 – ~2015	Power wall hit; performance shifted to multiple cores	~20%/yr (per-core nearly flat)
Post multicore / DSA	2015 – present	Domain-specific accelerators (GPUs, TPUs, NPUs), specialization	Workload-dependent, often very high for target workloads

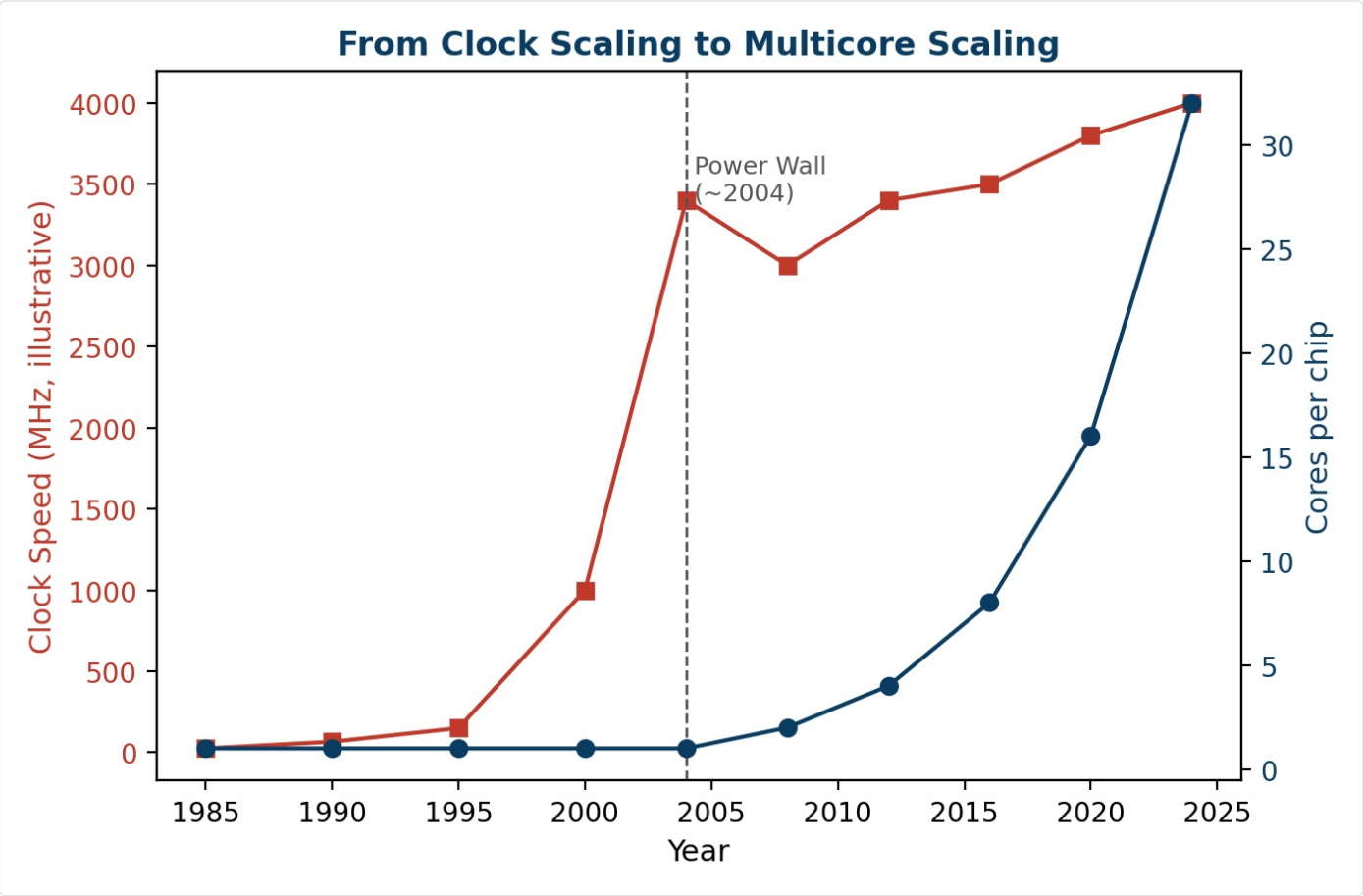


Fig 1.3 — Illustrative trend: clock speed growth flattens sharply after ~2004 (the "power wall"), while the number of cores per chip keeps climbing instead — the industry's practical response to the same underlying transistor budget.

Underlying technologies that architects must continuously track, because each evolves at a different rate:

- **Integrated circuit logic technology** — transistor density improves ~35–40%/year historically; die size grows only slowly.
- **DRAM (main memory) technology** — capacity per chip grows fast, but *access latency* improves much more slowly than logic/processor speed. This growing gap between processor speed and memory speed is precisely why Unit 2 (Memory Hierarchy) exists as an entire independent topic.
- **Magnetic/Flash storage** — capacity per dollar improves fastest of all technologies, but access latency (especially rotating magnetic disk) lags badly behind processor speed by many orders of magnitude.
- **Network bandwidth** — has also historically scaled quickly, changing how distributed/parallel and cloud systems are designed.

> **Key idea to remember for exams:** The single most important technology trend is that *processor speed improves much faster than memory speed* ("the memory wall"). This speed gap is the central motivation for the entire memory-hierarchy unit that follows — without caches, a fast CPU would spend most of its time simply waiting for data.

1.3 Classification of Parallel Computers (Flynn's Taxonomy)

Michael Flynn (1966) classified computer architectures based on the multiplicity of **Instruction streams** and **Data streams** that can be handled simultaneously by the machine. This remains the standard first classification taught in every computer architecture course because it cleanly separates "how many different things is the control unit doing" from "how many different data items is the hardware crunching."

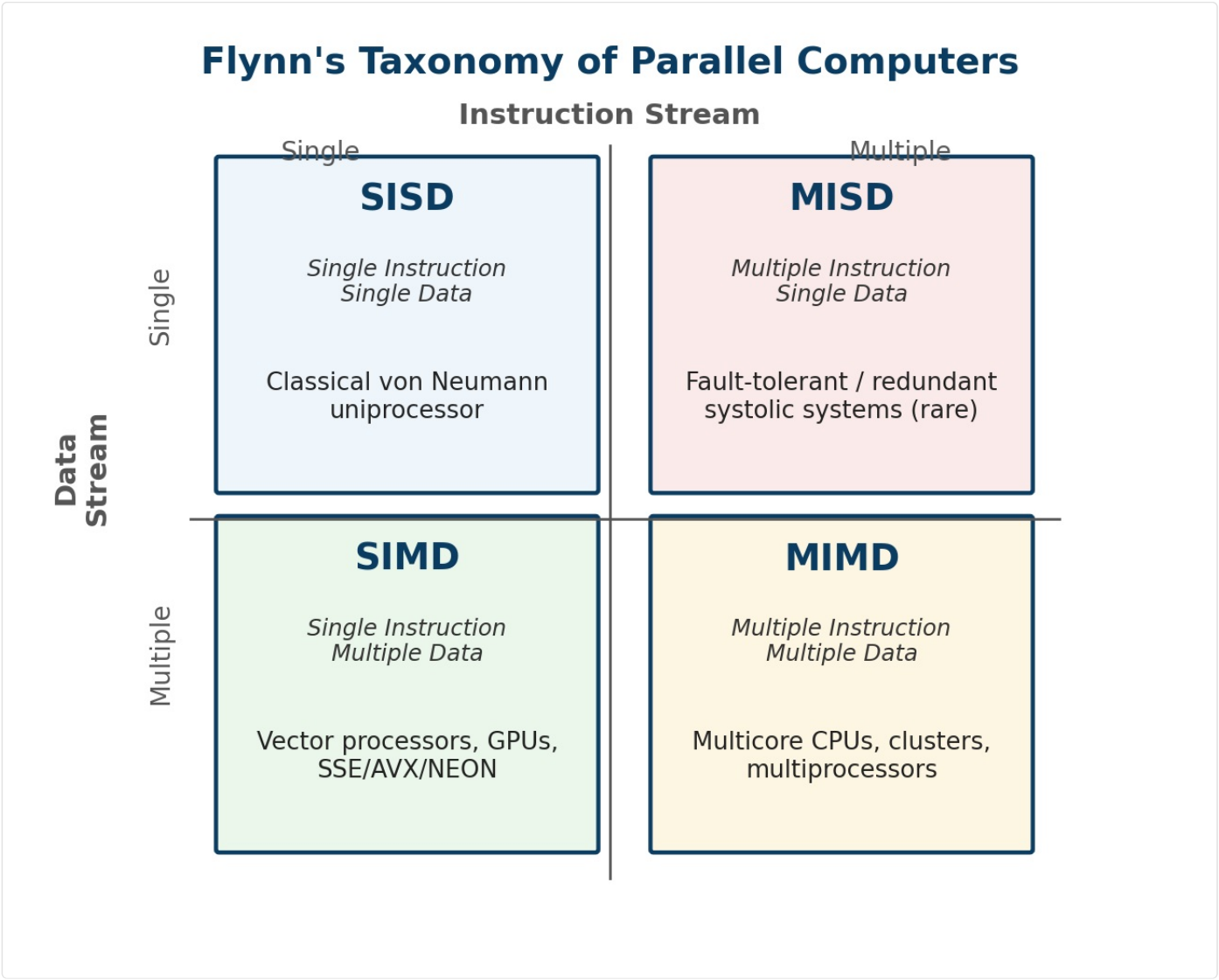


Fig 1.4 — Flynn's Taxonomy: the four fundamental classes of computer architecture, based on instruction-stream and data-stream multiplicity.

(a) SISD — Single Instruction, Single Data

> One control unit fetches one instruction stream that operates on one data stream at a time — one operation, on one piece of data, per step.

Example: Classical von Neumann uniprocessor (traditional single-core CPU without any explicit parallel execution model). Even a pipelined single-core superscalar CPU is still logically SISD from the programmer's point of view, because it still appears to execute one sequential instruction stream — the pipelining/superscalar tricks are internal organization, invisible at the ISA level.

(b) SIMD — Single Instruction, Multiple Data

> A single instruction is broadcast to many processing elements (PEs), each operating on its *own* data element simultaneously — "lock-step" execution.

Examples: Vector processors (Cray-1 and successors), array processors, GPUs (in their core streaming-multiprocessor execution model), and modern CPU SIMD extensions such as SSE/AVX (Intel/AMD) or NEON (ARM).

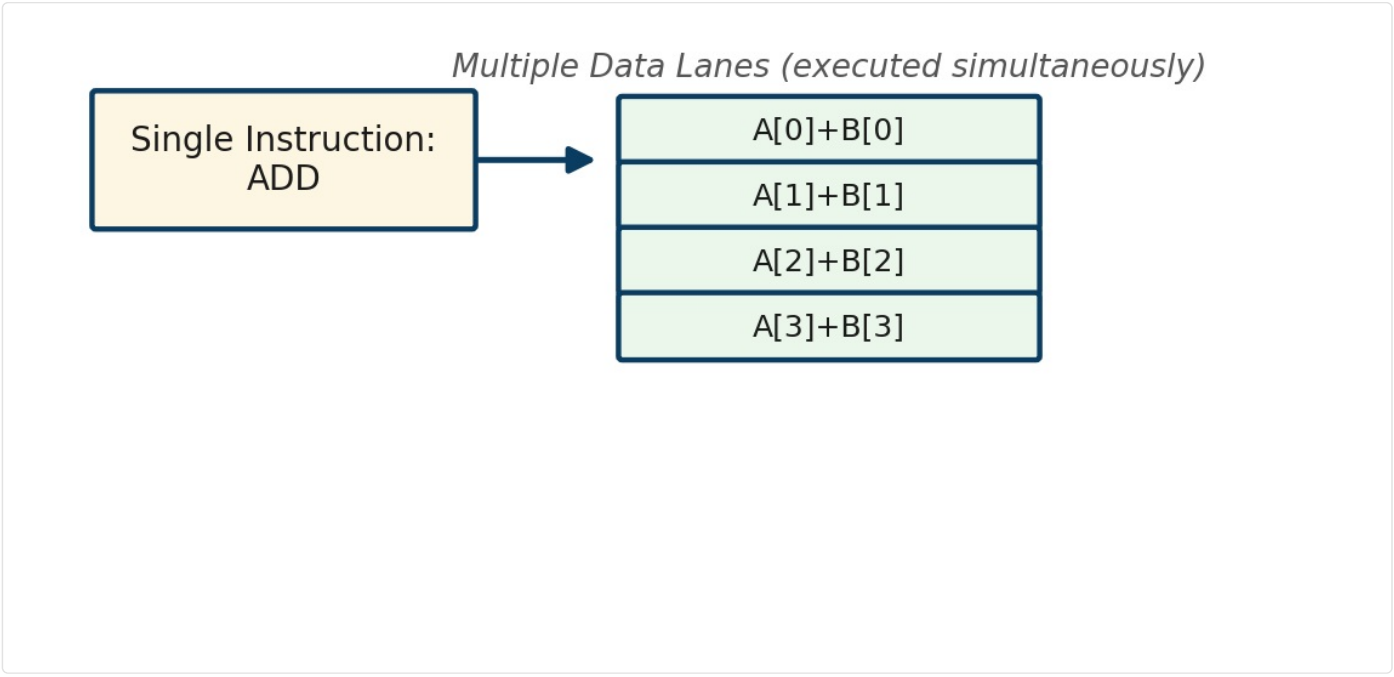


Fig 1.5 — SIMD execution: one ADD instruction is applied to four independent data lanes at once, instead of looping four separate times.

Example: Adding two arrays A[0..7] and B[0..7] element-wise. A SIMD unit with 8 lanes can execute the single instruction "ADD" on all 8 pairs in one clock cycle instead of looping through 8 scalar additions — a direct, hardware-level 8× throughput improvement for this kind of data-parallel work.

(c) MISD — Multiple Instruction, Single Data

> Multiple processing units execute different instructions on the *same* data stream simultaneously.

Rare in general-purpose computing; mainly seen in specialized fault-tolerant systems (e.g., space-shuttle flight-control computers running the same sensor data through different independently-written algorithms/hardware, then voting on the result to catch design or hardware faults) and certain systolic-array pipeline configurations. Most textbooks note MISD as "theoretically defined but practically uncommon."

(d) MIMD — Multiple Instruction, Multiple Data

> Multiple autonomous processors simultaneously execute different instructions on different data — the most general, and today by far the most common, form of parallel computer.

MIMD systems are further divided by how memory is organized and shared:

- **Shared-Memory MIMD:** all processors share one global, common address space; any processor can directly read/write any memory location.
  - **UMA** (Uniform Memory Access) — every processor sees the same access latency to every memory location (e.g., classic symmetric multiprocessors, SMP, connected via a shared bus or crossbar).
  - **NUMA** (Non-Uniform Memory Access) — each processor has "local" memory that it can access faster than "remote" memory physically attached to other processors — used in large multi-socket servers to keep scaling beyond what a single shared bus can support.
- **Distributed-Memory MIMD:** each processor has its own private, local memory; there is no shared address space, so processors communicate explicitly by sending messages over an interconnection network (e.g., compute clusters, MPI-based supercomputers).

Exam tip: Multicore CPUs, computing clusters and most modern supercomputers are MIMD machines; GPUs are best described as SIMD (or the vendor-specific refinement "SIMT" — single instruction, multiple threads). A frequently asked exam question is simply to classify a given real machine (a smartphone SoC, a GPU, a Beowulf cluster, an old vector supercomputer) into one of the four Flynn categories — practise this actively.

1.4 Performance-Based Computing

"Performance" sits at the very centre of computer architecture — nearly every design decision made anywhere in the stack is ultimately a performance/cost/power trade-off. Two common, and sometimes conflicting, performance measures matter to different stakeholders:

- **Response time (latency):** the total time between the start and completion of one task — this is what matters to an individual interactive user (e.g., "how long until my query returns?").
- **Throughput (bandwidth):** the total amount of work completed per unit time — this is what matters from a data-centre / server-farm perspective (e.g., "how many queries per second can this server handle?").

Optimizing purely for one can sometimes hurt the other — e.g., batching requests together can improve throughput while making each individual request's response time longer.

```
Performance(X) = 1 / ExecutionTime(X)
```

```
"X is n times faster than Y"  ⇒  Performance(X)/Performance(Y) = ExecutionTime(Y)/ExecutionTime(X) = n
```

The CPU Performance Equation

The execution time of a program can be decomposed into three independent factors, each controlled by a different part of the system — this decomposition is arguably the single most important formula in the whole unit, because it tells an architect exactly where to focus effort.

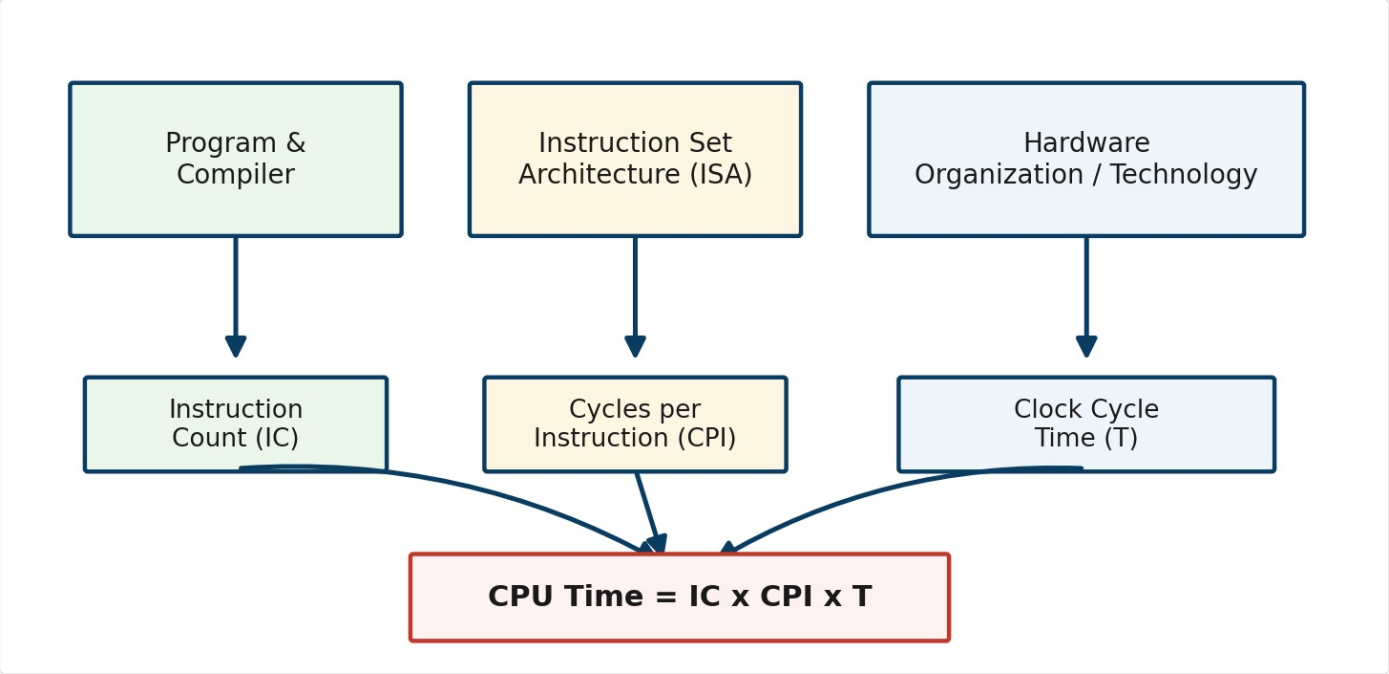


Fig 1.6 — The CPU performance equation: execution time results from the multiplication of three largely independent factors, each owned by a different layer of the system.

```
CPU Time = Instruction Count (IC) × Cycles Per Instruction (CPI) × Clock Cycle Time (T)
```

Equivalently, since Clock Rate = 1/T:

```
CPU Time = (IC × CPI) / Clock Rate
```

Factor	Depends primarily on
Instruction Count (IC)	The program/algorithm itself, compiler quality, and the ISA (how compactly the algorithm compiles down to machine instructions).
CPI (Cycles per Instruction)	The ISA and the micro-architecture — pipelining, hazards, and especially cache/memory stalls (Unit 2) add directly to average CPI.
Clock Cycle Time	Fabrication/circuit technology and pipeline depth (more pipeline stages generally allow a shorter cycle time, but raise CPI due to hazards).

Worked Example 1: A program has IC = 10^9 instructions, average CPI = 1.5, and clock rate = 2 GHz (so T = 0.5 ns).
CPU Time = (10^9 × 1.5) / (2×10^9) = 0.75 s.
If the compiler is improved so IC drops to 0.9×10^9 with the same CPI and clock rate, the new CPU time = $(0.9\times10^9\times1.5)/(2\times10^9)$ = 0.675 s → a speedup of 0.75/0.675 = 1.11×.

Worked Example 2 (weighted CPI): Suppose a program mix is 40% ALU instructions (CPI=1), 30% Load/Store (CPI=4), 20% Branch (CPI=2), 10% Other (CPI=3).
Average CPI = 0.4(1) + 0.3(4) + 0.2(2) + 0.1(3) = 0.4 + 1.2 + 0.4 + 0.3 = 2.3 cycles/instruction.
This weighted-CPI technique is a very common short-numerical exam question — always multiply each instruction class's *fraction* by its own CPI, then sum.

1.5 The "Myopic View" of Computer Architecture

A "myopic" (near-sighted) view of computer architecture is one that looks *only* at the processor/ISA in isolation — judging a design purely by raw clock speed, transistor count, or instruction count, while ignoring the rest of the system stack that actually determines delivered, real-world performance.

Why this view is misleading: Real delivered performance depends on the *entire* system working together: compiler code-generation quality, operating-system scheduling and interrupt overhead, memory-hierarchy behaviour (hit rates, bus contention), I/O subsystem speed, and even the specific characteristics of the application/workload being run. A processor with a higher advertised clock rate can easily be *slower* in practice than one with a lower clock rate but a smarter branch predictor, a bigger cache, and a better compiler — the classic trap popularly called the "megahertz myth" (famously exploited in early-2000s marketing wars between Intel Pentium 4 and AMD Athlon processors).

A non-myopic (holistic) approach to architecture instead evaluates the full hardware/software interface end-to-end. This is exactly why the "quantitative approach" philosophy (Hennessy & Patterson's textbook title itself) insists on measuring real workloads and standardized benchmarks (Section 1.9) rather than judging chips by ISA feature lists or clock speed alone.

Exam angle: If asked to "explain the myopic view of computer architecture," structure your answer as: (1) define it as a narrow, ISA/clock-only view, (2) explain why it's misleading using the CPU performance equation (a myopic view ignores CPI and IC effects), (3) give the megahertz-myth example, (4) conclude that the correct approach is a holistic, benchmark-driven, full-stack evaluation.

1.6 Energy, Power and Cost

Power

$$\text{Dynamic Power} \propto \text{Capacitive Load} \times \text{Voltage}^2 \times \text{Frequency}$$

Because power grows with the *square* of voltage, architects historically reduced supply voltage together with feature size (Dennard scaling) to keep total power roughly constant even as transistor counts and clock frequency rose generation after generation. Once voltage scaling slowed down dramatically around 2004–2005 (voltage cannot be reduced below certain physical thresholds without transistors becoming unreliable), any further increase in frequency or transistor count directly and unavoidably increases power. This is the origin of the **"power wall"** introduced in Section 1.2 — it single-handedly ended the era of ever-increasing single-core clock speeds and pushed the entire industry toward multicore designs (many simpler/slower cores instead of one increasingly hot, fast core).

- **Dynamic power:** consumed while transistors are actively switching (charging/discharging their load capacitance) — proportional to how often and how fast switching happens.
- **Static (leakage) power:** consumed even when transistors are idle, due to unavoidable current leakage through the transistor — becomes proportionally much more significant at smaller process nodes (7 nm, 5 nm) where transistors are packed extremely close together.

Worked Example: If clock frequency increases by 20% and voltage is held constant, dynamic power increases by roughly 20% too (linear in frequency). But if voltage must also rise by 10% to support that higher frequency reliably, power increases by approximately $1.1^2 \times 1.2 \approx 1.45\times$, i.e., about 45% — illustrating why the voltage term dominates and why "just clock it faster" quickly becomes thermally unsustainable.

Cost

The cost of an integrated circuit is dominated by **die cost**, which itself depends heavily on manufacturing **yield** (explained fully in Section 1.8). Other contributing cost factors include packaging, testing, and the **"learning curve"** effect — the manufacturing cost of a given chip design falls over time as the fabrication process matures, defect density drops, and yield improves through accumulated process experience.

1.7 Dependability

As computer systems become larger, more complex, and are increasingly used for safety-critical or business-critical applications (banking, aviation, medical devices, cloud infrastructure), correct and continuous operation matters as much as raw speed. Dependability formalizes this using a chain of related concepts:

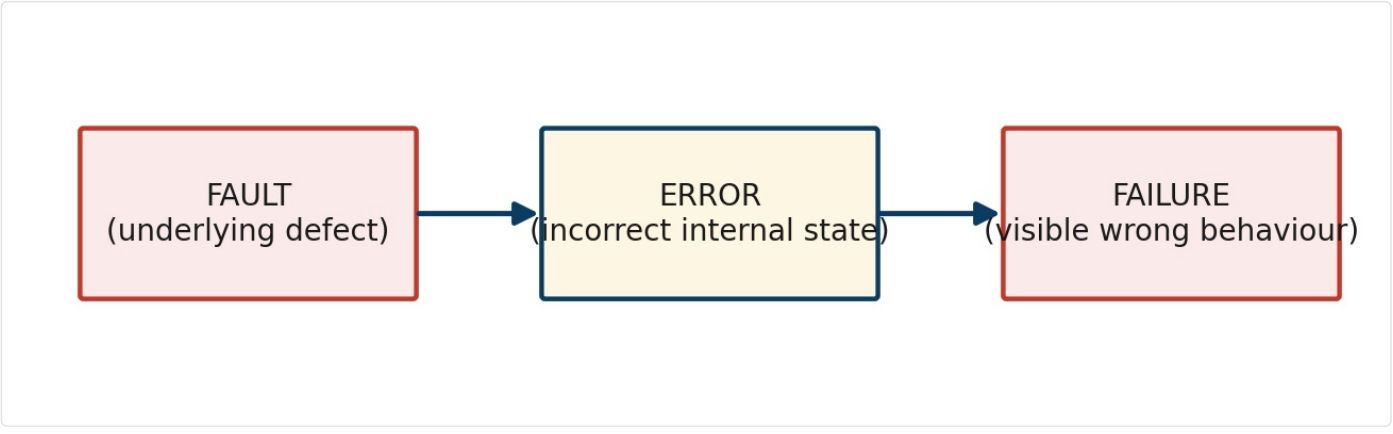


Fig 1.7 — The dependability chain: a latent fault may activate into an internal error, which may in turn propagate and become an externally visible failure.

A **Fault** (e.g., a manufacturing defect, a software bug, a cosmic-ray bit-flip) may, if activated, cause an **Error** (an incorrect internal state of the system). That error may, if it propagates to the system's output/interface, cause a **Failure** (the system deviates from its specified, expected behaviour, visible to the user). A fault does not always cause an error, and an error does not always cause a failure — good system design tries to catch and contain faults/errors before they become failures (this is the whole basis of fault-tolerant design, e.g., the MISD flight-computer example in Section 1.3).

Term	Meaning
Reliability	A measure of continuous correct service without interruption, quantified as **MTTF** (Mean Time To Failure).
MTTR	Mean Time To Repair — the average time taken to detect a failure and restore the system back to correct service.

MTBF	Mean Time Between Failures = MTTF + MTTR (the full failure-repair cycle length).
Availability	The fraction of total time that a system is actually usable/operational (as opposed to down for repair).

```
Availability = MTTF / (MTTF + MTTR) = MTTF / MTBF
```

Worked Example: A server has MTTF = 995 hours and MTTR = 5 hours.
Availability = 995 / (995+5) = 995/1000 = 0.995 → 99.5% availability.
Follow-up: if the same server's MTTR is halved (better/faster repair process) to 2.5 hours with MTTF unchanged, the new availability = 995/997.5 ≈ 99.75% — showing that improving *repair speed*, not just failure frequency, is a valid and often cheaper way to raise availability. This "two levers" idea (reduce failures OR reduce repair time) is a common short-answer exam point.

1.8 Processor Speed, Cost, Power Consumption and Fabrication Yield

Processor Speed

Determined jointly by clock rate and pipeline/micro-architectural organization — but, as the CPU performance equation (Section 1.4) makes clear, raw clock speed alone never determines actual delivered performance; average CPI (heavily influenced by memory-hierarchy stalls, the whole subject of Unit 2) is at least equally important, and often more so for modern workloads.

Fabrication Yield

Yield = the fraction of manufactured chips (dies) on a silicon wafer that come out fully functional (defect-free) after fabrication.

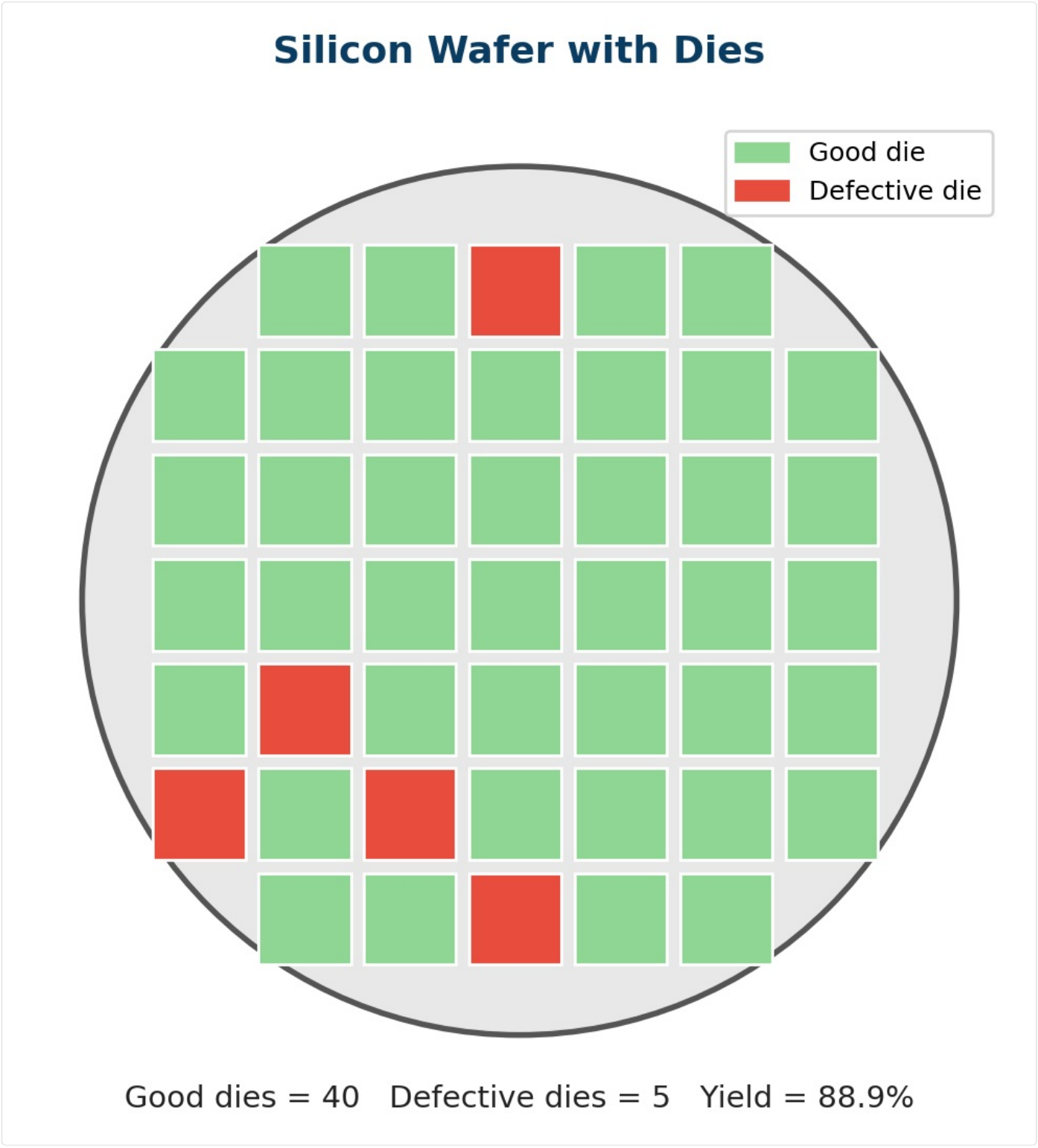


Fig 1.8 — A silicon wafer patterned into individual dies. Random manufacturing defects (red) scattered across the wafer make some dies non-functional; yield is simply the good-die fraction.

A simplified empirical yield model (a negative-binomial-style model commonly taught in architecture courses):

$$\text{Die Yield} = \text{Wafer Yield} \times 1 \,/\, (1 + (\text{Defects per unit area} \times \text{Die Area}))^{N}$$

where N is a process-complexity parameter (typically between 11.5 and 15.5 for modern processes). Key relationships an architect must internalise:

- **Larger die area → lower yield** (a bigger die has a proportionally higher chance of overlapping at least one random defect) → higher effective cost per good die.
- **Cost per die = (Cost per wafer) / (Dies per wafer × Yield)** — this single relationship is why cutting-edge, very large, very dense processor dies are disproportionately expensive: yield falls in a highly non-linear way as die size grows, so cost rises faster than area alone would suggest.

> **Worked numerical intuition:** If doubling a die's area causes yield to fall from, say, 80% to 55% (a realistic non-linear drop for large dies), the number of usable good dies obtained per wafer does not simply halve — it falls by roughly a factor of 2 × (80/55) ≈ 2.9×, meaning cost-per-good-die nearly triples, not just doubles. This exact insight — *doubling die area more than doubles cost* — is why modern high-end processors are increasingly built as several smaller "chiplets" connected together rather than one giant monolithic die (better aggregate yield economics).

Power Consumption (Processor-Design Context)

Because a processor's total power budget is capped by practical cooling limits and (for mobile/laptop devices) battery/thermal design power (TDP)

limits, designers must trade off clock frequency against the number of cores and the per-core micro-architectural complexity. This trade-off is a direct, practical consequence of the dynamic-power formula in Section 1.6, and is the concrete engineering reason why MIMD multicore architectures (Section 1.3) became the dominant design strategy industry-wide after roughly 2004.

1.9 Performance Metrics, Benchmarks and Performance Laws

Problems with Simple Metrics

- **MIPS** (Million Instructions Per Second) — misleading because different instruction sets/compilers do different amounts of "real work" per instruction; a RISC machine naturally executes more, simpler instructions than a CISC machine for the same task, so comparing raw MIPS across different ISAs is meaningless.
- **MFLOPS** (Million Floating-Point Operations per Second) — useful only for floating-point-heavy scientific/numerical code; says nothing about general-purpose workloads dominated by integer, branch, or memory operations.

> The only fully reliable way to compare two machines is to time actual, representative *real programs / workloads* running on both — this practical necessity is exactly what motivates standardized benchmark suites.

Benchmark Standards

Benchmark Suite	Purpose
SPEC (CPU, INT/FP)	Industry-standard CPU benchmark suite built from real applications (separate integer & floating-point programs) — the most widely cited suite in both academic research and hardware marketing.
TPC (TPC-C, TPC-H, etc.)	Transaction Processing Performance Council — benchmarks specifically for database/OLTP transaction throughput, used heavily for server/enterprise hardware comparisons.
Whetstone / Dhrystone	Older, simpler synthetic benchmarks for floating-point and integer performance respectively; largely superseded by SPEC today but still referenced historically.
Kernels	Small, heavily-used code fragments extracted directly from real programs (e.g., matrix multiply, FFT, sort) — useful for isolating a specific bottleneck rather than whole-program behaviour.

When combining scores across many benchmark programs, architects use the **geometric mean** of normalized performance ratios (relative to a fixed reference machine), never a simple arithmetic mean — the geometric mean is the only aggregation method that gives consistent, order-independent relative comparisons regardless of which machine is chosen as the reference.

Amdahl's Law

> Amdahl's Law gives the theoretical maximum overall speed-up of a task when only *part* of it can be improved or parallelized — the *unimproved (serial)* fraction eventually dominates total execution time and strictly limits total achievable speedup, no matter how much faster the improved portion is made.

```
Speedup = 1 / [ (1 − F) + F/S ]
where F = fraction of original execution time that benefits from the enhancement, S = speedup achieved on that enhanced fraction alone.
```

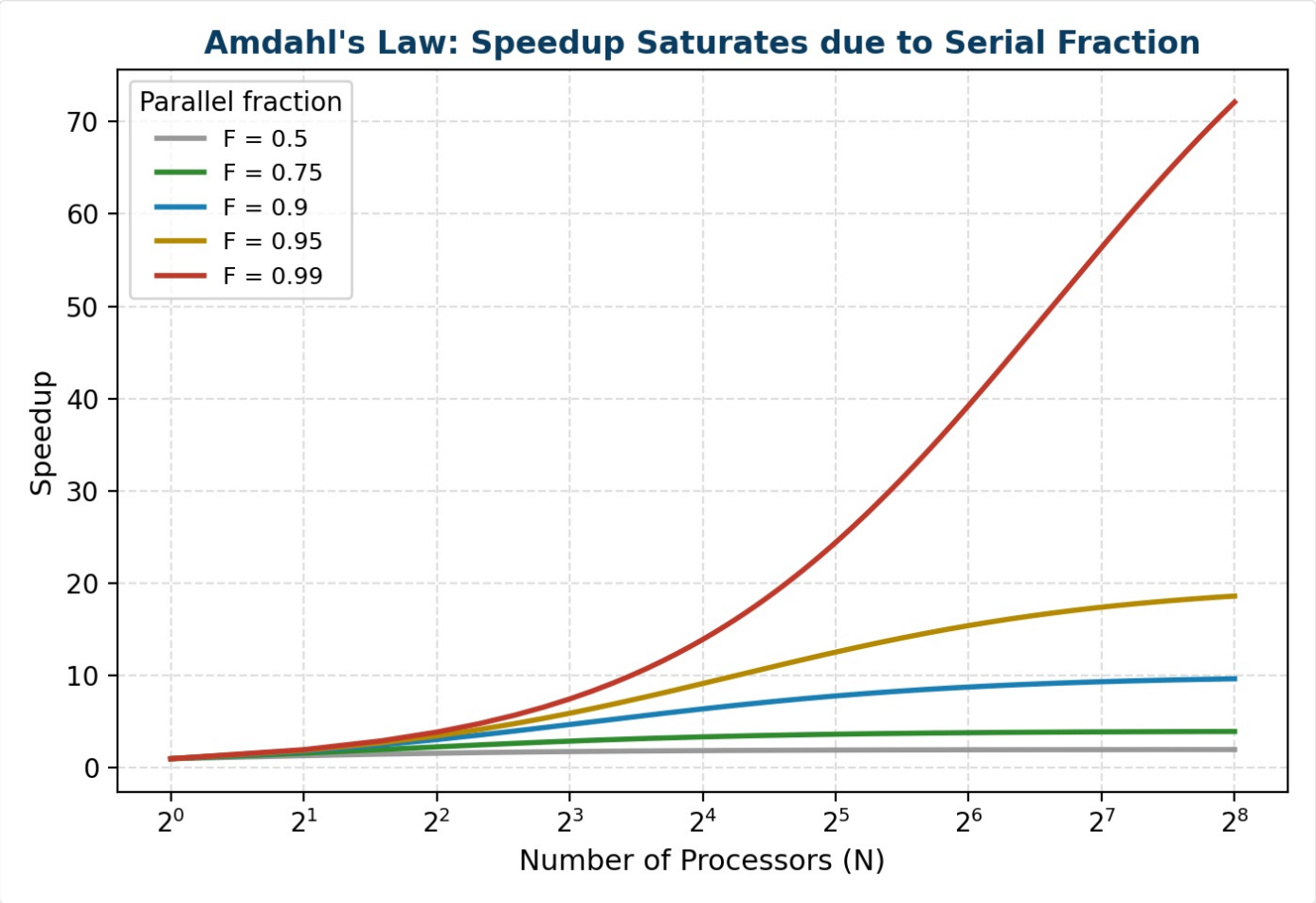


Fig 1.9 — Amdahl's Law: for a fixed problem size, speedup always saturates to a hard ceiling of 1/(1−F) as the number of processors N grows, no matter how large N becomes.

Worked Example: Suppose 80% (F = 0.8) of a program can be parallelized, and enough processors are used to get a 10× speedup on that portion alone (S = 10).
Speedup = 1 / [(1−0.8) + 0.8/10] = 1 / [0.2 + 0.08] = 1/0.28 ≈ 3.57×.
Notice that even with infinite processors (S → ∞), the maximum possible speedup is capped at 1/(1−F) = 1/0.2 = 5× — the serial 20% alone caps everything else that can ever be achieved.

Key exam takeaway: Amdahl's Law shows that focusing purely on making the parallel portion faster gives sharply diminishing returns past a point; the sequential (non-parallelizable) fraction is the true long-run bottleneck. This is the standard, strongest argument for why reducing the serial portion of an algorithm matters more, in the long run, than simply adding more processors.

Gustafson's Law (listed in the syllabus table)

Note: the official syllabus table lists this topic as "*Lhadma's Law*" immediately next to Amdahl's Law; this is the standard companion result universally taught alongside Amdahl's Law in exactly this position in every computer-architecture course, and is known everywhere in the literature as **Gustafson's Law** (also called Gustafson–Barsis's Law). It is presented below under its correct, standard name so you can study and cite it accurately.

Gustafson's Law takes the opposite viewpoint from Amdahl: instead of holding the problem size fixed and simply adding more processors, it assumes that as more processors become available, users naturally choose to solve *correspondingly bigger problems* in the same fixed time budget — which sidesteps Amdahl's pessimistic fixed-size ceiling entirely.

```
Scaled Speedup = (1 − F) + F × N
where N = number of processors, F = parallel fraction (measured on the scaled-up, parallel system, not the original serial one).
```

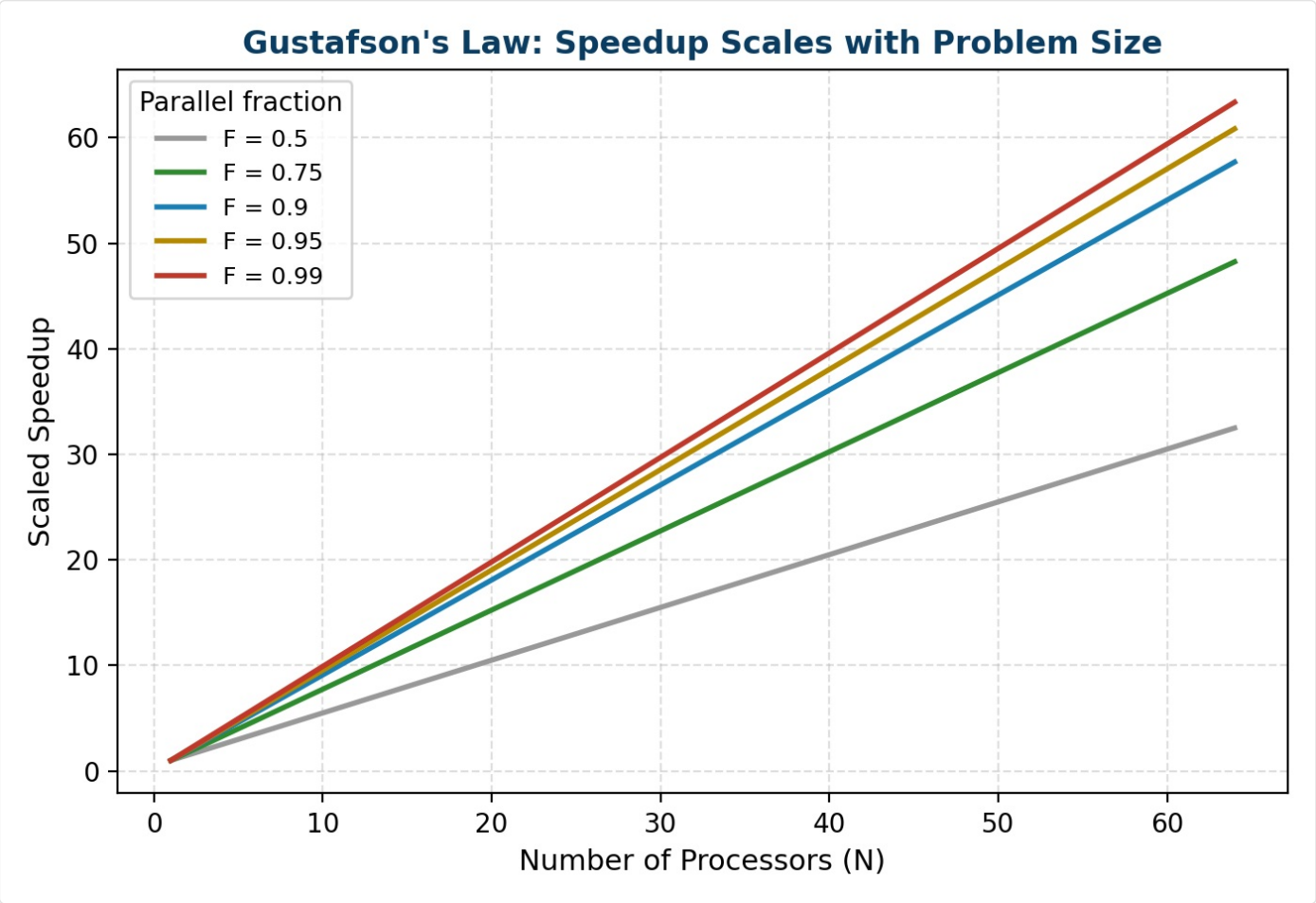


Fig 1.10 — Gustafson's Law: when the problem size is allowed to scale with the number of processors, speedup grows almost linearly with N instead of saturating.

Worked Example: With F = 0.8 (80% parallel fraction) and N = 10 processors:
Scaled Speedup = (1−0.8) + 0.8×10 = 0.2 + 8 = 8.2× — a far more optimistic figure than Amdahl's fixed-size prediction for the same F, precisely because here the total workload itself is allowed to grow proportionally with the number of processors available.

Aspect	Amdahl's Law	Gustafson's Law
Problem size	Fixed	Scales up with processor count
Underlying view	Pessimistic — speedup is strictly bounded	Optimistic — near-linear speedup is achievable
Best suited for	Fixed, latency-critical workloads (e.g., real-time tasks)	Large-scale scientific/data-parallel computing where bigger machines are used to solve bigger problems

Measuring Performance — Summary Principles

- Always benchmark using representative, real workloads — never rely on synthetic microbenchmarks alone.
- Use the geometric mean to fairly aggregate normalized performance ratios across an entire benchmark suite.
- Report both response time and throughput, since different designs can favour one over the other.
- Use Amdahl's Law to reason about the upper bound on speedup achievable from any single optimization — this is a recurring, high-value numerical exam question, so practise several variants (solving for F, for S, or for required N given a target speedup).

UNIT 2 : Memory Hierarchy Design

2.1 Basics of Memory Hierarchy

No single memory technology is simultaneously fast, large, cheap per byte, and low-power — every real technology forces a trade-off along these axes. The **memory hierarchy** solves this problem by organizing memory into multiple levels of increasing size (and decreasing speed/cost per byte), exploiting the fact that programs do not access all of their data uniformly at random, but instead concentrate their accesses (Section 2.2).

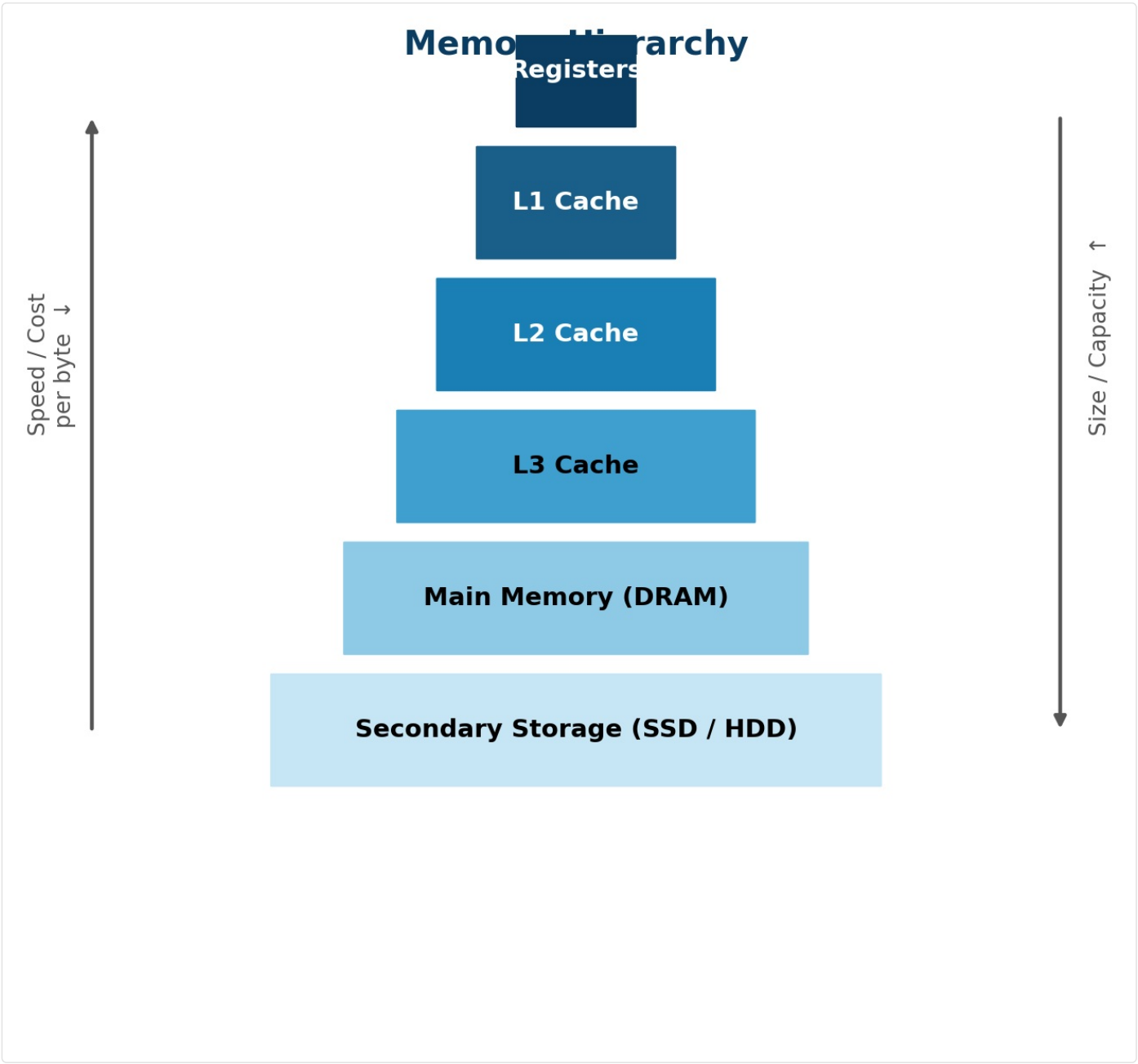


Fig 2.1 — The memory hierarchy pyramid. Levels near the top are small, extremely fast, and expensive per byte; levels near the bottom are huge, slow, and cheap per byte.

> **Goal of the hierarchy:** To give the programmer/CPU the illusion of a memory that is as fast as the fastest (top) level and as large/cheap as the largest (bottom) level — achieved in practice by automatically keeping the most frequently and recently used data in the faster, smaller levels closest to the CPU.

Each level in the hierarchy effectively acts as a "cache" for the level immediately below it (main memory can even be thought of as a cache for disk, in the context of virtual memory — Section 2.7). The whole design succeeds specifically because of the locality of reference exhibited by essentially all real programs.

Key Hierarchy Parameters

Parameter	Meaning
Hit	Requested data is found already present in the current (faster) level.
Miss	Requested data is not found in the current level; it must be fetched from a lower (slower) level.
Hit time	Time required to access data that is already present in this level.

Miss penalty	The extra time needed to fetch data from the next lower level following a miss, on top of the hit time.
Hit ratio / Miss ratio	The fraction of all memory accesses that result in a hit / a miss, respectively (they sum to 1).

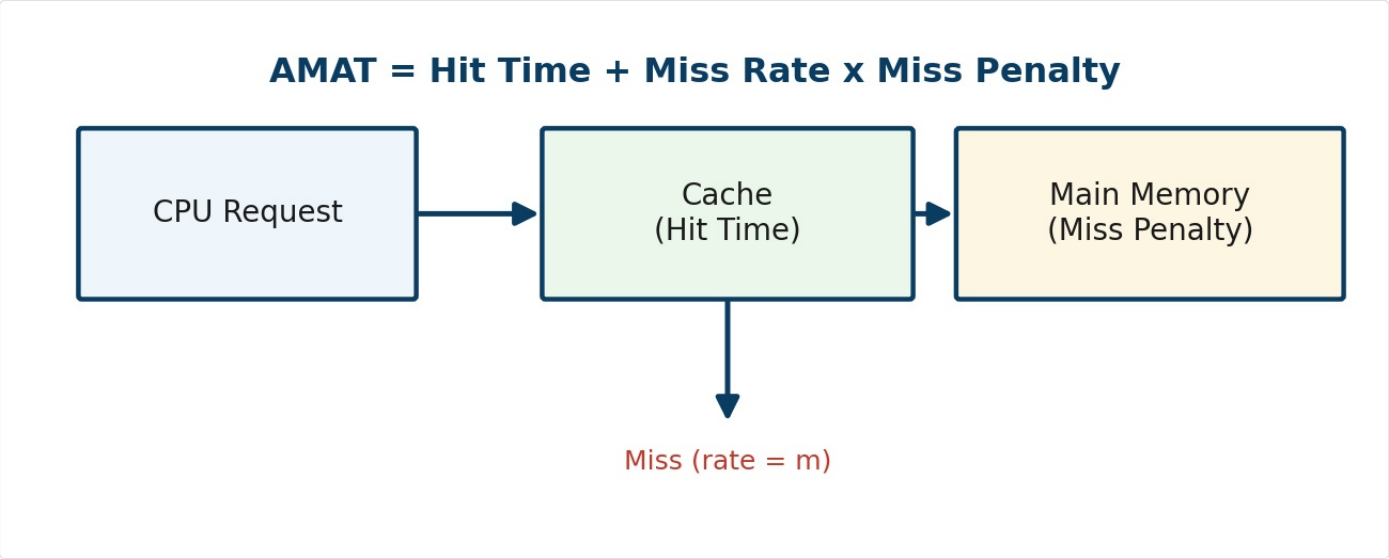


Fig 2.2 — Average Memory Access Time (AMAT): most accesses are satisfied quickly by the cache (hit time); the remaining fraction (miss rate) pays an additional, much larger miss penalty to reach main memory.

```
Average Memory Access Time (AMAT) = Hit Time + Miss Rate × Miss Penalty
```

Worked Example 1: L1 hit time = 1 cycle, miss rate = 5%, miss penalty = 20 cycles.
AMAT = 1 + 0.05×20 = 1 + 1 = 2 cycles.

Worked Example 2 (multi-level): L1 hit time = 1 cycle, L1 miss rate = 8%. On an L1 miss, L2 is checked: L2 hit time = 10 cycles, L2 miss rate (local) = 20%, and an L2 miss costs a further 100-cycle penalty to reach main memory.
AMAT = L1 Hit Time + L1 Miss Rate × (L2 Hit Time + L2 Miss Rate × Memory Access Time)
= 1 + 0.08 × (10 + 0.20 × 100) = 1 + 0.08 × 30 = 1 + 2.4 = 3.4 cycles.
This nested-AMAT calculation for multilevel caches is a very common longer-format numerical exam question — always work from the innermost (fastest) level outward.

2.2 Inclusion, Coherence and Locality Properties

Kai Hwang's treatment of memory hierarchy formally defines three fundamental properties that together justify why a multi-level hierarchy works both *correctly* (never silently loses or corrupts data) and *efficiently* (actually speeds programs up in practice):

(a) Inclusion Property

Data present in a smaller/faster (inner) level of the hierarchy must also be present in every larger/slower (outer) level. i.e., Cache ⊂ Main Memory ⊂ Disk.

This guarantees that if a piece of data is not found in the cache, it can always be located at some outer level — no data item is ever "only" in the fast cache and nowhere else (the one carefully-managed exception is a "dirty," not-yet-written-back block under a write-back policy, which is handled explicitly rather than being allowed to violate the property silently).

(b) Coherence Property

All copies of the same data item, across different levels of the hierarchy (or across different processor caches in a multiprocessor system), must eventually be kept consistent — i.e., an update made at one level/copy must be properly propagated so that no other level/copy ever supplies stale, outdated data.

Two basic update strategies exist at the single-processor cache-to-memory level:

- **Write-through:** every write to the cache is immediately also written through to main memory → memory is always kept perfectly consistent with the cache, but this generates significantly more memory-bus traffic.
- **Write-back:** writes update only the cache; the modified ("dirty") block is written back to memory later, typically only when it is evicted from the cache → far less memory traffic, but requires careful bookkeeping using a per-block "dirty bit."

In multiprocessor systems, this exact same underlying idea extends to keeping multiple *different processors' private caches* consistent with each other — this harder, distributed version of the coherence problem is covered in full depth in Section 2.6.

(c) Locality of Reference Property

Real programs tend to access only a relatively small portion of their total address space at any given time window — this strong predictability

is precisely what makes caching effective at all; without locality, caches would provide little to no benefit.

- **Temporal Locality:** if a memory location is accessed once, it is likely to be accessed again soon afterward (e.g., loop control variables, a frequently-called function's code, a hot global variable).
- **Spatial Locality:** if a memory location is accessed, nearby memory locations are likely to be accessed soon after (e.g., sequential array traversal, sequential instruction fetch of straight-line code).

Example: In the loop `for(i=0;i<n;i++) sum += A[i];` — accessing `A[i]` exhibits strong *spatial* locality (the next element `A[i+1]` sits immediately adjacent in memory and will very likely be loaded into the same cache line already), while the variables `sum` and `i` exhibit strong *temporal* locality (both are re-used on literally every single iteration of the loop).

2.3 Cache Memory Organization

A cache is physically divided into fixed-size **blocks (lines)**. The mapping rule that decides where a given main-memory block is allowed to be placed within the cache defines the cache's organization type.

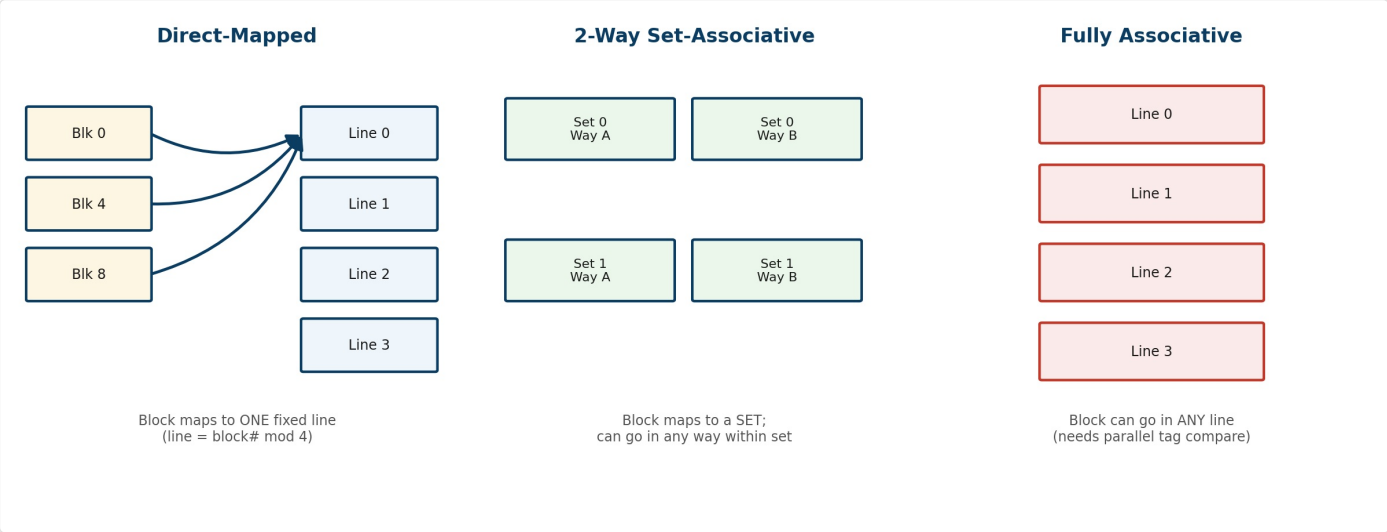


Fig 2.3 — The three fundamental cache-mapping schemes: direct-mapped (one fixed slot), set-associative (a small fixed group of slots), and fully associative (any slot at all).

(a) Direct-Mapped Cache

Each memory block maps to exactly **one** specific cache line, typically via: Cache Line Index = (Block Address) mod (Number of Cache Lines).

Pros: Very simple, fast, and cheap hardware (only a single tag comparison is ever needed, no parallel comparators). **Cons:** High conflict-miss rate — if two frequently-used blocks happen to map to the same line, they will repeatedly evict each other even though the rest of the cache sits completely unused ("thrashing").

(b) Fully Associative Cache

A memory block may be placed in **any** line of the cache whatsoever. This requires comparing the requested tag against every single line's tag simultaneously, using parallel comparator hardware.

Pros: Minimizes conflict misses entirely — a block is only ever evicted due to genuine capacity pressure, never due to an unlucky mapping collision. **Cons:** Expensive and power-hungry hardware (needs one comparator per line, all operating in parallel), and typically slower to access — practical mainly for small structures such as TLBs (Section 2.7), not large main caches.

(c) Set-Associative Cache

A practical middle ground: the cache is divided into a number of "sets"; each memory block maps to exactly one specific set (just like direct mapping), but within that set it may occupy *any* one of a small number of lines ("ways") — like limited-scope full association. An "n-way set-associative" cache has exactly n lines ("ways") per set.

Set index is computed as: **Set Index = (Block Address) mod (Number of Sets)**. Most real-world processors today use set-associative caches (very commonly 4-way or 8-way) as the practical sweet-spot compromise between hit rate, access speed, and hardware cost/power.

Cache Write Policies

Policy	Description
Write-through	Every write updates both cache and main memory simultaneously (simpler logic, but generates significantly more memory traffic).
Write-back	Writes update only the cache; the block is marked "dirty" and is written to memory only later, on eviction (much faster on average, but needs a dirty-bit per block).

Write-allocate	On a write-miss, the target block is first loaded into the cache, and the write is then applied to the newly-loaded cached copy.
No-write-allocate (write-around)	On a write-miss, the write is sent directly to main memory, completely bypassing the cache (useful when the written data is unlikely to be re-read soon).

Cache Replacement Policies

Needed whenever a set (or the whole cache, if fully associative) is completely full and a newly-fetched block must displace an existing one (relevant only for associative and set-associative caches — direct-mapped caches have no choice to make, since each block has exactly one possible line):

- **LRU (Least Recently Used):** evicts the block that has gone unused for the longest time — directly exploits temporal locality, and typically gives the best hit rate in practice, but is costlier to implement with perfect exactness in hardware as associativity grows.
- **FIFO (First-In-First-Out):** evicts whichever block was loaded into the cache the longest time ago, completely regardless of how recently it was actually used.
- **Random:** evicts a uniformly-randomly chosen line from the set — surprisingly effective in practice for higher associativities, and very cheap and simple to implement in hardware (no tracking state required at all).

Types of Cache Misses ("The 3 C's")

Miss Type	Cause
Compulsory (Cold)	The very first-ever access to a given block — this kind of miss is fundamentally unavoidable regardless of how large or clever the cache is.
Capacity	The cache simply cannot hold every block needed during the program's execution at once, even assuming perfect (full) associativity and an ideal replacement policy.
Conflict	Multiple actively-used blocks compete for the same limited set/line (this class of miss occurs only in direct-mapped and set-associative caches — it cannot occur at all in a fully associative cache).

2.4 Advanced Optimization of Cache Performance

The following techniques, drawn primarily from Hennessy & Patterson, attack AMAT by specifically reducing hit time, miss rate, or miss penalty — a good exam answer should always name *which one of the three* a given technique targets.

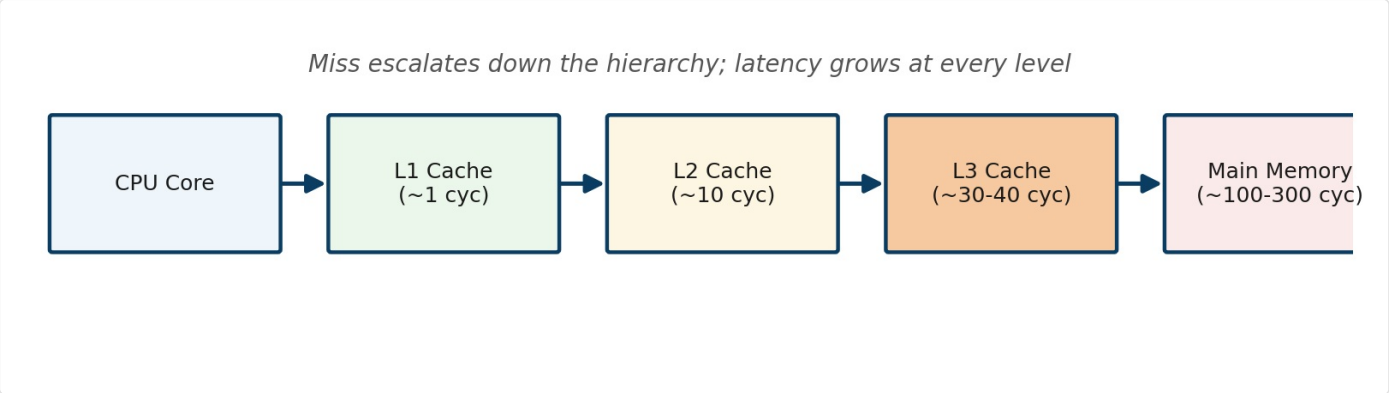


Fig 2.4 — A modern multilevel cache hierarchy: L1 is fastest but tiny, L2 and L3 are progressively larger and slower, and only a genuine L3 miss must pay the full, very large main-memory latency.

Technique	What it targets	How it helps
Larger block size	Miss rate	Exploits spatial locality more aggressively; but if made too large it increases miss penalty (more data to transfer per miss) and can waste memory bandwidth on unused data.
Larger cache size	Miss rate	Directly reduces capacity misses; but increases hit time (larger structures are inherently slower to search) and raises hardware cost/power.
Higher associativity	Miss rate	Reduces conflict misses by giving each block more possible homes; slightly increases hit time due to the extra parallel tag comparisons required.
Multilevel caches (L1/L2/L3)	Miss penalty	A fast, small L1 backed by a larger, slower L2/L3 means most L1 misses are resolved quickly in L2 rather than paying the full round-trip cost to main memory every single time.
Priority to read misses over writes	Miss penalty	Reads are usually on the processor's critical path (the CPU is typically stalled waiting for the result), whereas writes can safely be buffered and completed slightly later without stalling the pipeline.
Avoiding address translation during cache indexing	Hit time	Using a virtually-indexed, physically-tagged cache design allows the TLB lookup (Section 2.7) and the cache-array access to proceed in parallel rather than serially.
Way prediction	Hit time	A small predictor guesses in advance which specific "way" of a set-associative cache holds the requested data, avoiding the need to check every way in parallel every single time.

Trace caches	Hit time / IPC	Stores already-decoded dynamic instruction traces (used in some high-performance superscalar CPUs, e.g., Pentium 4) to avoid the cost of repeatedly re-decoding the same instructions.
Non-blocking (lockup-free) caches	Miss penalty	Allows the cache to keep servicing further hits — and even further independent misses — while an earlier miss's fetch from memory is still in flight ("hit under miss" / "miss under miss").
Hardware prefetching	Miss rate	Dedicated hardware observes the ongoing access pattern (e.g., a simple sequential stream) and automatically fetches blocks into the cache before they are ever explicitly requested by the program.
Compiler-controlled prefetching	Miss rate	The compiler inserts explicit prefetch instructions well ahead of a load/store it statically predicts will be needed soon.
Compiler optimizations (loop interchange, blocking/tiling)	Miss rate	Restructures the order of memory accesses in the source code / loop nest itself to better match cache-line and cache-capacity boundaries, maximizing both spatial and temporal locality without changing program correctness.

Loop Interchange Example: Accessing a 2-D array stored in row-major order as `for j: for i: A[i][j]` jumps across entire rows on every single access (very poor spatial locality — each access likely misses). Simply interchanging the loop order to `for i: for j: A[i][j]` instead accesses memory perfectly sequentially — this dramatically improves the cache hit rate with absolutely zero change to the program's logical output.

Loop Blocking (Tiling) Example: For large matrix multiplication, naively looping over the full matrices causes rows/columns to be evicted from cache before they can be reused. "Blocking" splits the matrices into small sub-blocks (tiles) that are each small enough to fit entirely within the cache, and completes all the work on one tile before moving to the next — dramatically increasing temporal-locality reuse and often producing dramatic (multi-times) real-world speedups on large scientific computing workloads.

2.5 Memory Technology and Optimization

Technology	Characteristics
SRAM (Static RAM)	Used for on-chip caches and registers. Very fast (no periodic refresh needed since the stored bit is held in a stable latch), but low density and expensive — typically 6 transistors per stored bit.
DRAM (Dynamic RAM)	Used for main memory. Much higher density and far cheaper per bit (just 1 transistor + 1 capacitor per bit), but significantly slower, and requires periodic refresh cycles because the storage capacitor slowly leaks its charge over time.
Flash memory	Non-volatile (retains data with no power); used in SSDs. Slower to write than DRAM and has a limited number of write/erase cycles (requiring wear-levelling algorithms), but is much faster than magnetic disk and has no moving parts.
Magnetic Disk	Very high capacity per dollar, non-volatile, but suffers from mechanical latency (seek time + rotational delay) that makes it many orders of magnitude slower than DRAM — this enormous gap is exactly what makes virtual memory's page-fault penalty so severe (Section 2.7).

DRAM Optimizations

- **SDRAM (Synchronous DRAM):** operation is synchronized to the system clock, which allows pipelined/burst-mode access patterns instead of one slow independent access at a time.
- **DDR SDRAM (Double Data Rate):** transfers data on both the rising *and* the falling edge of the clock signal, doubling effective bandwidth without needing to raise the underlying clock frequency at all (and its later generations DDR2/DDR3/DDR4/DDR5 push this further with higher internal parallelism).
- **Memory banking / interleaving:** splits memory into several independent banks that can be accessed concurrently or in an overlapped pipelined fashion, substantially increasing effective aggregate bandwidth.
- **Wider buses / burst mode:** transfers several consecutive words per single memory request, amortizing the relatively large fixed row-activation overhead over more useful data.

2.6 Cache Coherence and Synchronization Mechanisms

The Cache Coherence Problem

In a multiprocessor system, each processor typically has its own private cache. If two (or more) processors happen to cache the same memory block, and one of them modifies its own copy, the other processors' cached copies silently become stale/outdated unless an explicit protocol actively keeps them consistent — this is the **cache coherence problem**, a direct multiprocessor extension of the single-processor coherence property from Section 2.2.

Formal coherence requirements a correct protocol must guarantee

- A read by processor P to location X, occurring after a write by that same P to X (with no other writes to X in between), must return the value P itself just wrote.
- A read by any processor to location X, occurring sufficiently after a write to X by a *different* processor, must return that written value (assuming no other intervening writes).
- Writes to the very same location made by different processors must be observed in the exact **same order** by every processor in the system

("write serialization") — otherwise different processors could disagree about the final value.

Coherence Protocol Approaches

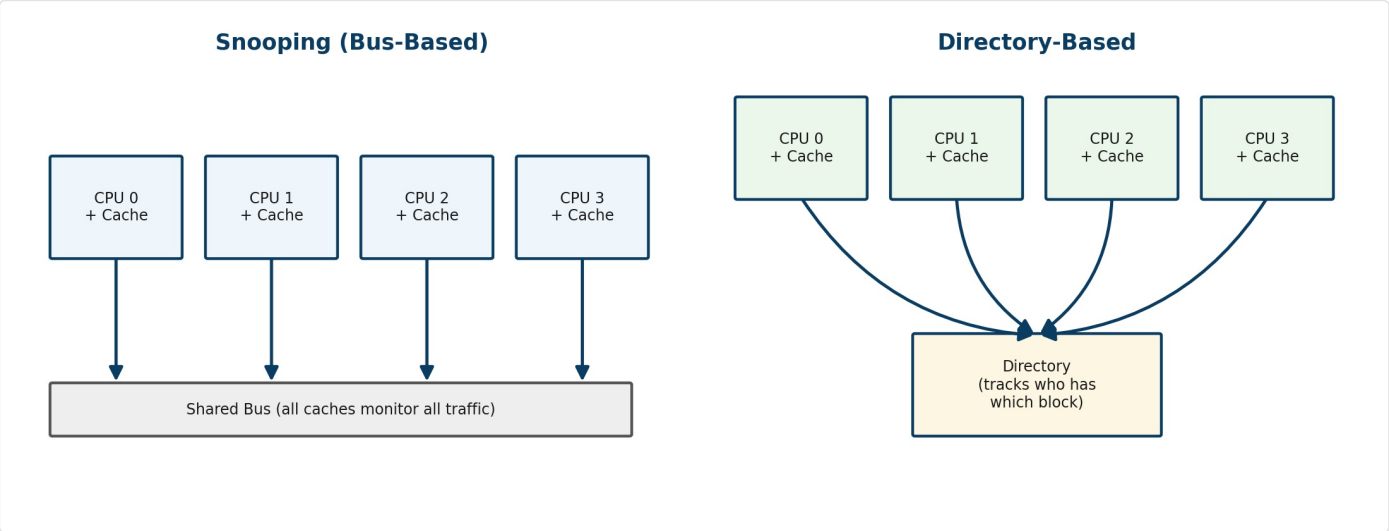


Fig 2.5 — Two fundamentally different ways to keep multiple processor caches coherent: snooping (every cache watches every bus transaction) versus a directory (a central record of who currently holds each block).

Approach	Description
Snooping protocols	All caches continuously monitor ("snoop on") a shared broadcast bus; every single cache watches every other cache's read/write transactions and reacts locally (invalidating or updating its own copy) as needed. Works well and is simple for small-scale, bus-based systems, but broadcasting to everyone does not scale to large processor counts.
Directory-based protocols	A central (or physically distributed) "directory" structure explicitly tracks exactly which caches currently hold a copy of each block; coherence messages are then sent only to the specific caches known to actually hold a copy, rather than being broadcast to everyone — this targeted approach scales much better to large multiprocessors where broadcasting every transaction would overwhelm the interconnect.

Common Snooping Protocol States — MSI / MESI

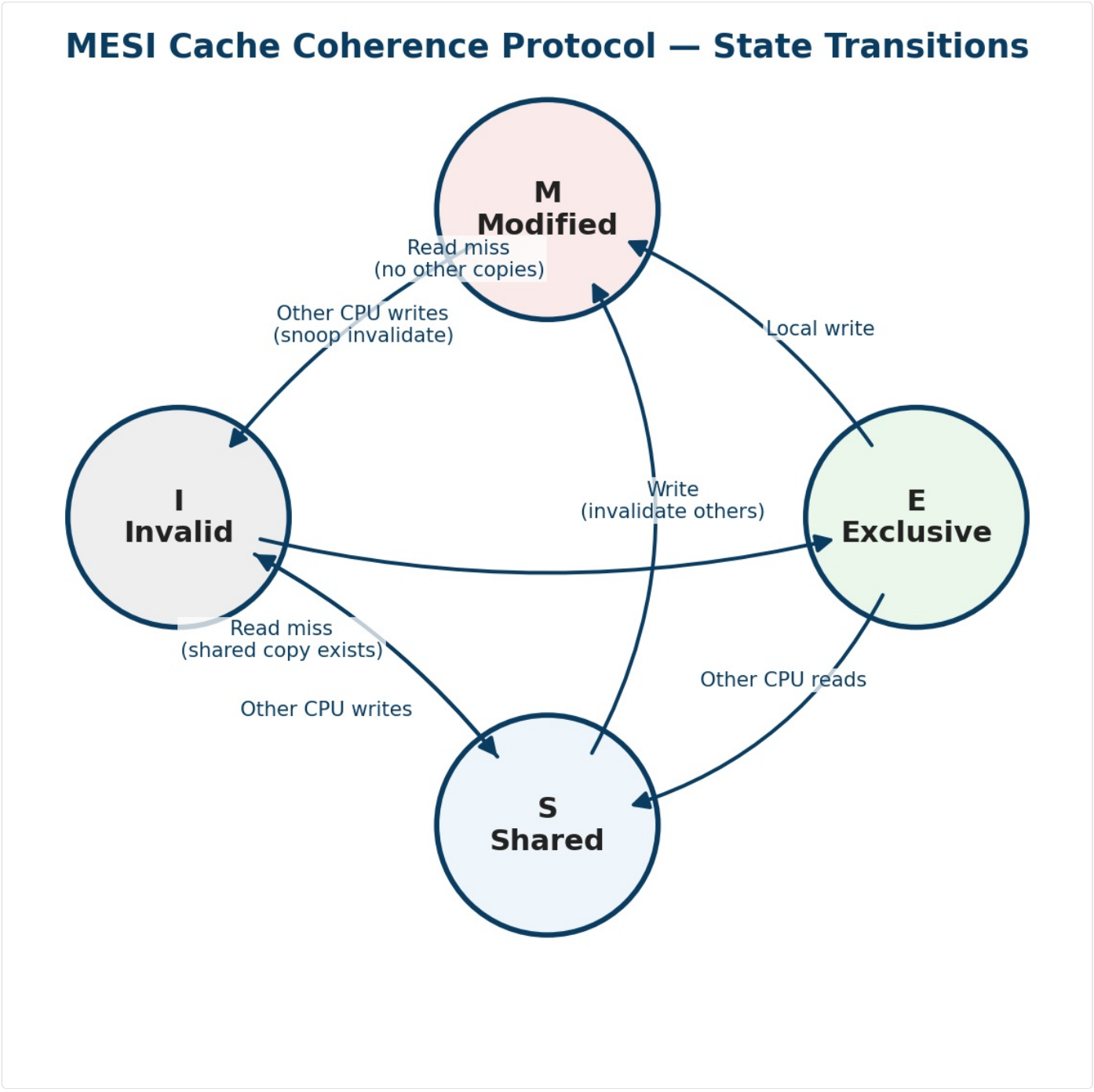


Fig 2.6 — The MESI protocol's state-transition diagram. Every cached block in every processor's cache is always in exactly one of these four states, and transitions between them are triggered by local reads/writes or by snooped remote reads/writes.

State	Meaning
M — Modified	The block is modified in this cache only; the copy in main memory is now stale; this cache alone must supply the correct data if any other processor requests it.
S — Shared	The block is unmodified and may exist simultaneously in multiple caches, with all copies consistent with main memory.
I — Invalid	The block is not valid in this cache at all (either it was never loaded, or it was invalidated because another processor wrote to it).
E — Exclusive (MESI only, not present in plain MSI)	The block is present in only this one cache, is unmodified, and matches main memory — this extra state lets the cache transition silently to Modified on a local write, without needing any bus transaction at all, which is the key optimization MESI adds over plain MSI.

On any write by any processor, every other cache currently holding a copy of that block must transition its copy to the Invalid state (in invalidation-based protocols such as MESI) — this single mechanism is what actively enforces the three formal coherence requirements listed above.

Synchronization Mechanisms

Multiprocessor systems also need low-level hardware primitives to safely coordinate concurrent access to shared data structures by multiple simultaneously-running threads/processors:

- **Test-and-Set:** an atomic (indivisible) instruction that reads a memory location and unconditionally sets it to 1, all in a single indivisible hardware step — the classic textbook building block for implementing simple spin-locks.
- **Compare-and-Swap (CAS):** atomically compares a memory location's current value against an expected value and, only if they match, swaps in a new value — this is the foundational primitive underlying most modern lock-free data structures.

- **Load-Linked / Store-Conditional (LL/SC):** a cooperating pair of instructions used together to implement an atomic read-modify-write sequence without requiring one single monolithic atomic instruction — the store only succeeds if no other processor wrote to that location in between the paired load and store.
- **Locks / Mutexes and Semaphores:** higher-level software synchronization constructs, built on top of the hardware primitives above, used to enforce mutual exclusion and to safely coordinate producer-consumer style access patterns between threads.
- **Barriers:** synchronization points at which every participating thread/processor must arrive and wait before any single one of them is allowed to proceed further — commonly used to cleanly synchronize the distinct phases of a parallel algorithm (e.g., ensuring all processors finish phase 1 before any starts phase 2).

2.7 Virtual Memory

> Virtual memory is a memory-management technique that gives each running process the illusion of a large, private, contiguous address space of its very own — completely independent of the actual amount of physical DRAM installed in the machine — by transparently mapping virtual addresses to physical addresses (and further out to disk, when necessary).

Why Virtual Memory?

- **Isolation / Protection:** each process gets its own private address space, which prevents one buggy or malicious process from ever directly reading or corrupting another process's memory.
- **Simplifies programming:** application programmers do not need to manually manage physical memory placement, nor worry about exactly how much physical RAM happens to be installed on the machine they're running on.
- **Efficient, controlled sharing:** allows deliberate, controlled sharing of specific physical pages (e.g., shared system libraries loaded once, mapped into many processes) between otherwise-isolated processes.
- **Extends usable memory:** allows programs whose total combined memory footprint actually exceeds installed physical RAM to still run correctly, using disk as a transparent overflow area ("swap space").

Paging

> The virtual address space is divided into fixed-size **pages**; physical memory is correspondingly divided into equal-sized **frames**. A per-process **page table** maps each virtual page number to whichever physical frame number currently holds it (if any).

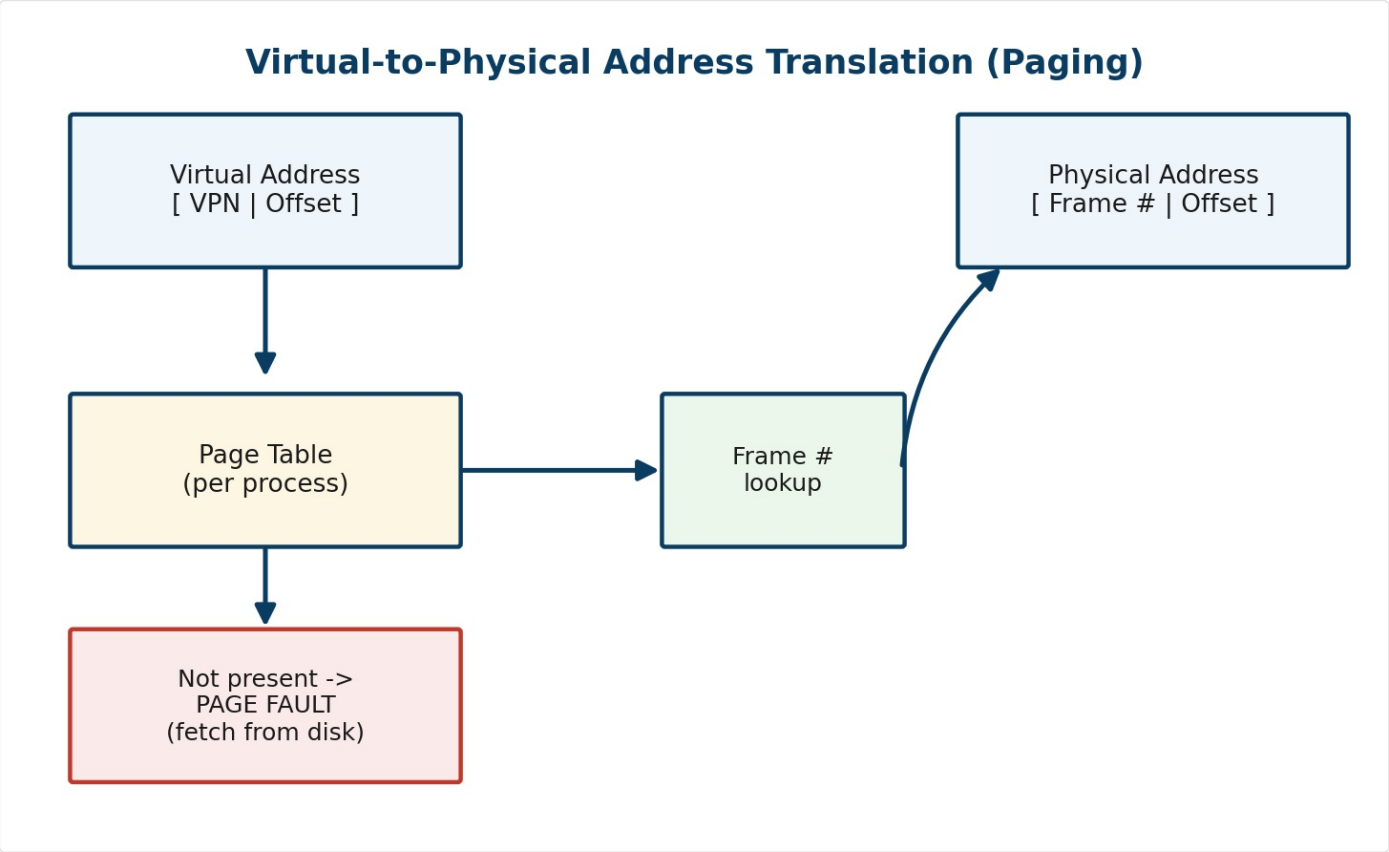


Fig 2.7 — Virtual-to-physical address translation via paging: the virtual page number is looked up in the page table to find the corresponding physical frame, while the page offset passes through unchanged.

Page Faults

> A **page fault** occurs when a process accesses a virtual page that is not currently resident in physical memory — the operating system must then fetch the required page from disk, possibly first evicting some other page to make room (chosen using a page-replacement policy such as LRU), then correctly update the page table, and finally resume the faulting instruction as if nothing had happened.

Because handling a page fault requires an actual disk access, its penalty is enormous — commonly measured in *milliseconds*, compared to a cache

miss penalty measured in mere *nanoseconds* (a difference of roughly five to six orders of magnitude). This huge asymmetry is exactly why page-replacement algorithms are chosen very carefully, favouring prediction *accuracy* heavily over decision speed — unlike ordinary cache-line replacement, where decision speed matters far more because the penalty for a wrong choice is comparatively tiny.

Translation Lookaside Buffer (TLB)

> Since consulting the full page table on literally every single memory access would be prohibitively slow, a small, extremely fast, typically fully-associative hardware cache called the **TLB** stores the most recently used virtual-to-physical page translations, avoiding a repeated full page-table walk on most accesses — a direct, textbook application of the locality principle introduced in Section 2.2.

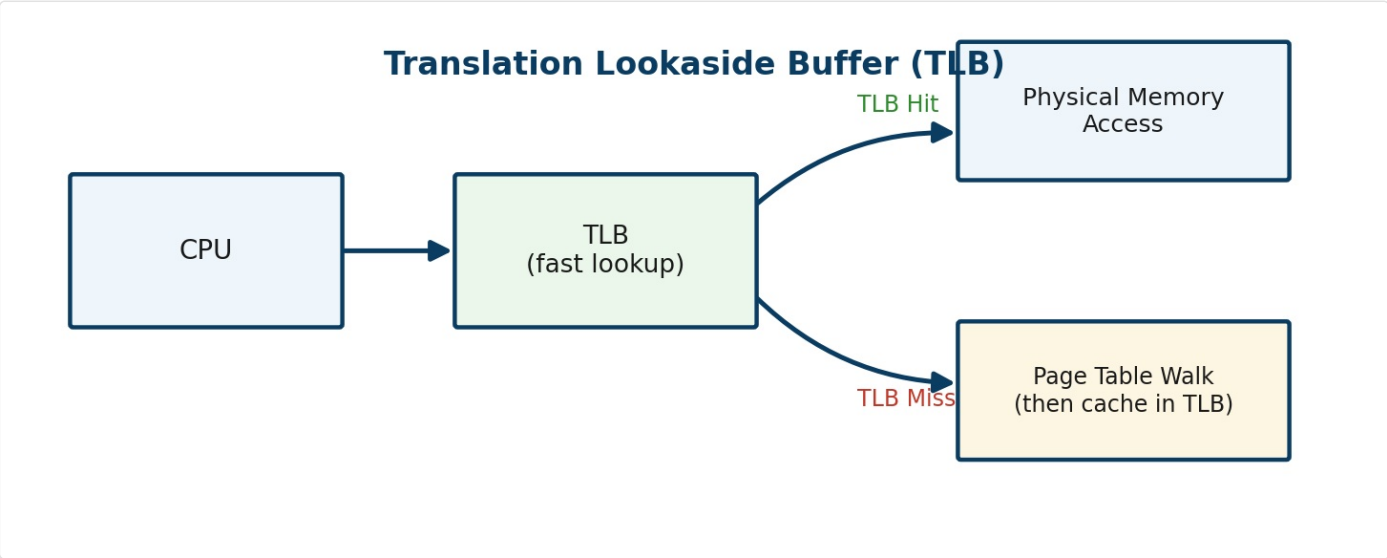


Fig 2.8 — The TLB sits between the CPU and physical memory: a TLB hit gives an almost-free translation, while a TLB miss forces a slower page-table walk whose result is then cached for next time.

TLB miss: requires performing a full "page table walk" to locate the correct mapping (done either in hardware or in software, depending on the specific ISA/architecture), after which the newly-found translation is inserted into the TLB so that future accesses to the same page hit quickly.

Segmentation (brief contrast)

An alternative (and sometimes complementary) memory-management scheme divides the address space into variable-sized logical **segments** (e.g., a code segment, a stack segment, a heap segment) rather than uniform fixed-size pages — this offers a more natural, logical view that directly matches program structure, but is considerably harder to manage efficiently than pure paging (it suffers from external fragmentation, since variable-sized free gaps of memory can accumulate over time). Many real systems historically combined both ideas as "paged segmentation" to get benefits of each.

2.8 Virtual Machines

> A Virtual Machine (VM) is software that creates a convincing abstraction of underlying hardware (or of an OS/application execution environment) so that multiple, mutually isolated "virtual" computers can run concurrently on a single piece of physical hardware.

Types of Virtual Machines

Type	Description	Example
System VM (Hardware-level)	Provides a complete virtual hardware environment on top of which an entire guest operating system can run unmodified; managed and enforced by a **hypervisor / Virtual Machine Monitor (VMM)**.	VMware, VirtualBox, Xen, KVM, Hyper-V
Process VM (Application-level)	Provides a single platform-independent execution environment for one individual program/process, abstracting away the specifics of the underlying host OS and ISA entirely.	Java Virtual Machine (JVM), .NET Common Language Runtime (CLR)

Hypervisor / VMM Types

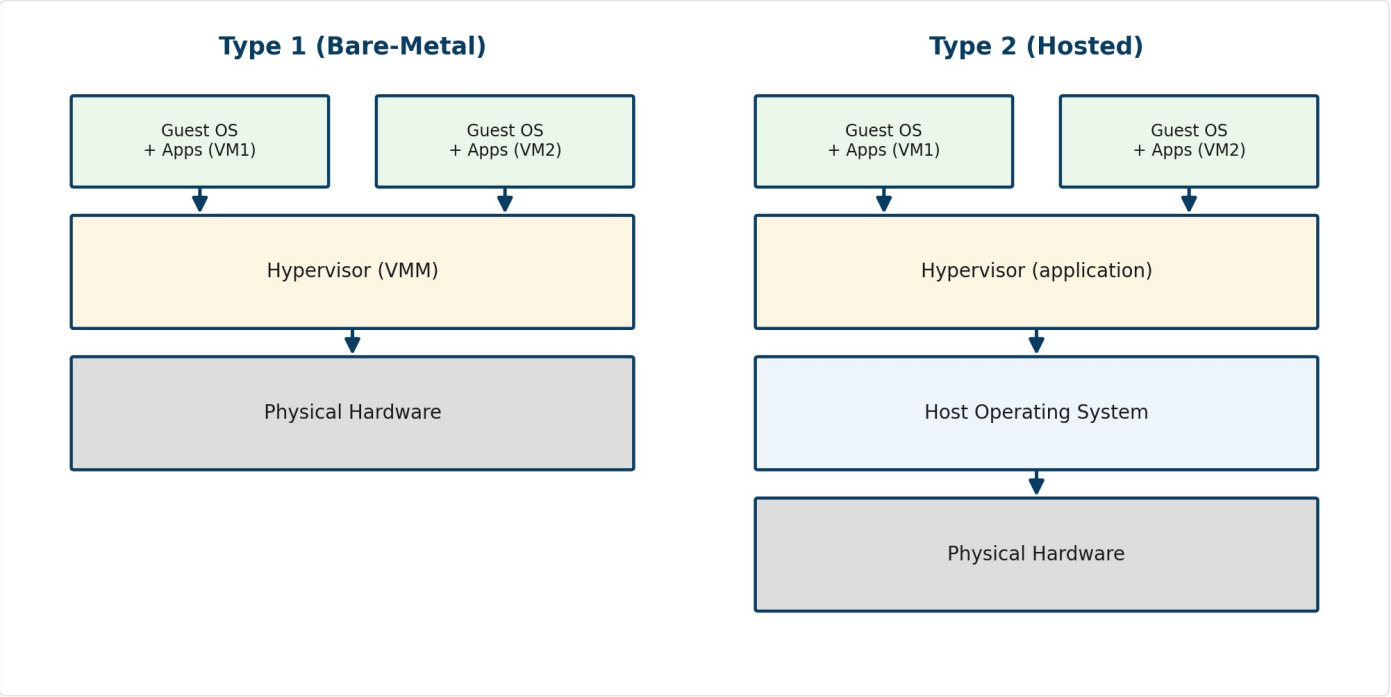


Fig 2.9 — Type 1 (bare-metal) hypervisors run directly on hardware for maximum efficiency in data centres; Type 2 (hosted) hypervisors run as an ordinary application on top of a conventional host OS, trading some efficiency for convenience.

- **Type 1 (Bare-metal / Native):** runs directly on the physical hardware itself, with no general-purpose host operating system underneath it at all — favoured in data centres and server environments specifically for its superior efficiency and minimal overhead (e.g., Xen, VMware ESXi).
- **Type 2 (Hosted):** runs as an ordinary application on top of a conventional, already-installed host operating system (e.g., VirtualBox or VMware Workstation running on top of Windows or Linux) — more convenient for desktop/development use, at some cost in raw efficiency.

Requirements for an Architecture to be Efficiently Virtualizable

(These are Popek and Goldberg's classical virtualization requirements, foundational to this entire topic):

- **Equivalence:** a program running under the VMM should behave identically to how it would behave running directly on the real underlying hardware (aside from possible differences in timing and available resources).
- **Resource control:** the VMM must retain complete, unconditional control over all virtualized hardware resources — guest software must never be able to bypass the VMM and access real hardware directly without permission.
- **Efficiency:** a statistically dominant fraction of guest instructions should be able to execute directly on the real hardware without any VMM intervention at all, for the virtualized system's performance to remain acceptable in practice.

Any instructions that could potentially break isolation if executed directly (so-called "privileged" or "sensitive" instructions) must instead trap into the VMM, so that it can emulate their effect safely on behalf of the guest. Architectures with a clean, well-defined separation between privileged and non-privileged instructions are dramatically easier to virtualize efficiently — this is precisely why dedicated hardware virtualization extensions such as Intel VT-x and AMD-V were introduced, adding an entirely new, more-privileged CPU mode reserved specifically for the hypervisor itself, sitting below the guest OS's own kernel mode.

Benefits of Virtualization

- Significantly higher hardware utilization — multiple independent workloads can efficiently share one single physical machine instead of each needing dedicated hardware.
- Strong fault and security isolation between different VMs running on the same physical host.
- Much easier system management overall: live migration of running VMs between physical hosts, point-in-time snapshots, and rapid provisioning of new "machines" on demand.
- Continued support for legacy software or older operating systems on modern hardware that could no longer run them natively.

Unit 2 — Summary Table

Concept	One-line takeaway
Memory Hierarchy	Multiple levels trade off speed vs. size vs. cost; the whole approach works only because of locality of reference.
Inclusion/Coherence/Locality	The three formal properties that together make a hierarchy both correct and genuinely effective.
Cache Organization	Direct-mapped (simplest), fully-associative (most flexible), set-associative (the practical real-world compromise).
Cache Optimization	Every technique attacks hit time, miss rate, or miss penalty specifically — never all three at once with one single trick.
Memory Technology	SRAM (fast, used for cache) vs. DRAM (dense, used for main memory) vs. Flash/Disk (huge capacity, non-volatile, but slow).
Cache Coherence	Essential in any multiprocessor; snooping/MESI suits small scale, directory-based protocols scale to large systems.
Virtual Memory	Gives every process the illusion of a large, private memory space via paging, page tables, and the TLB.

Virtual Machines	Abstract away hardware/OS specifics so that multiple isolated systems can safely share one physical machine.

Prepared study notes based on TCS 704 syllabus (Graphic Era University) — reference texts: Kai Hwang, "Advanced Computer Architecture" (3rd Ed.); Hennessy & Patterson, "Computer Architecture: A Quantitative Approach". All diagrams are original illustrations created for this document.